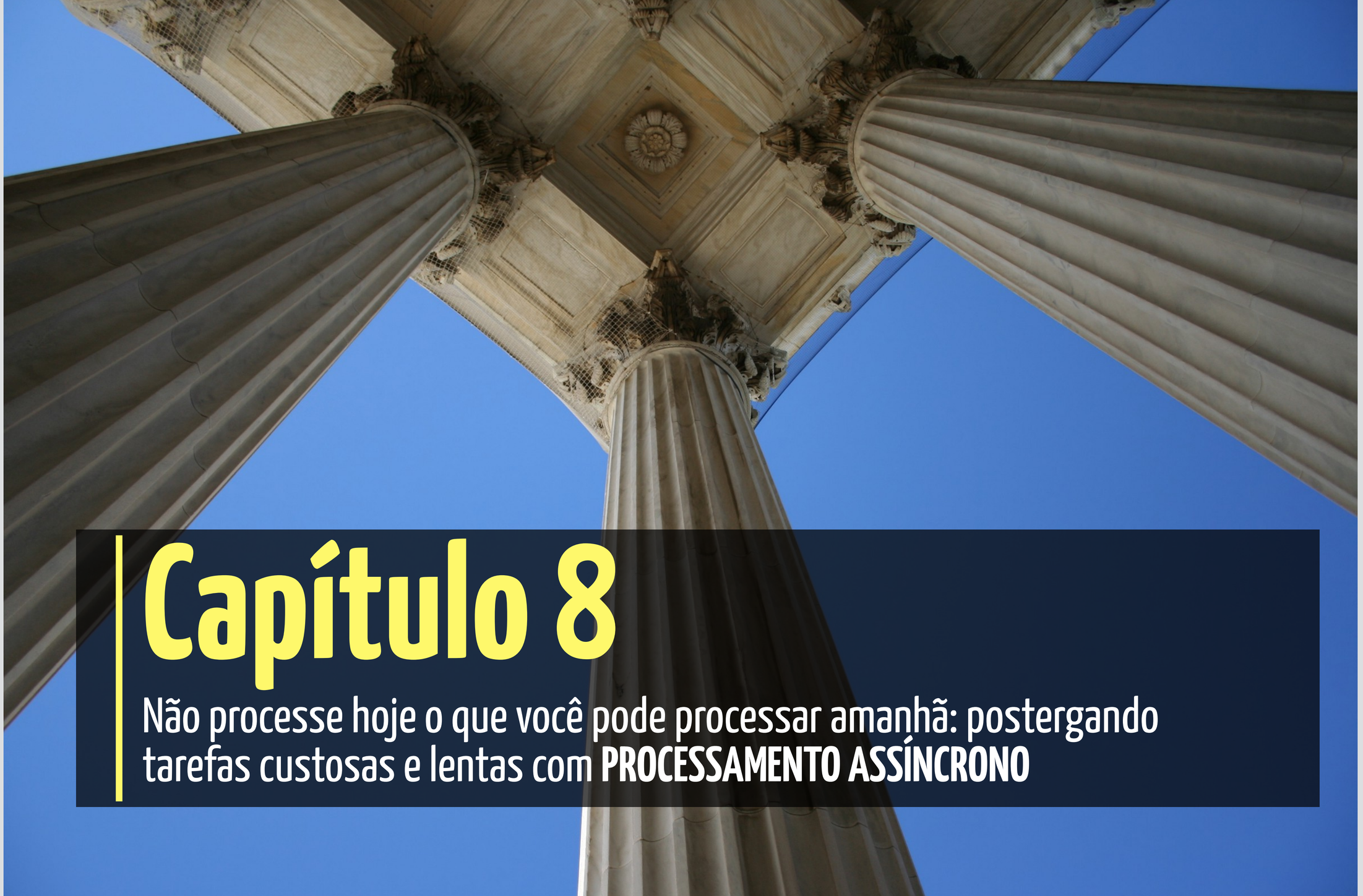
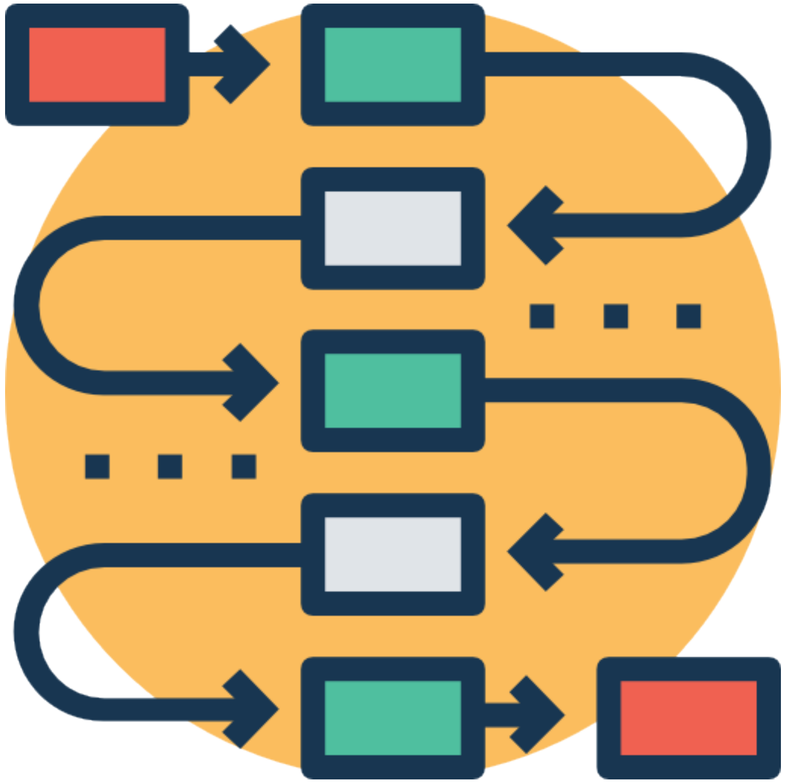


Capítulo 8

Não processe hoje o que você pode processar amanhã: postergando tarefas custosas e lentas com **PROCESSAMENTO ASSÍNCRONO**

Não subestime os PICOS de acesso em endpoints de escrita!

E se houver um
processamento **PESADO, LENTO**
ou **CUSTOSO**?



```java
@Controller
public class PedidoController {

    @Autowired
    public PedidoService service;

    @PostMapping("/vendas/finaliza")
    public void finalizaPedido(Pedido pedido) {
      // faz alguma validação de dados

      service.finaliza(pedido); // demorado...
    }
}
```

```java
@Controller
public class PedidoController {

    @Autowired
    public PedidoService service;

    @PostMapping("/vendas/finaliza")
    public void finalizaPedido(Pedido pedido) {
      // faz alguma validação de dados

      service.finaliza(pedido); // demorado...
    }
}
```

```java
@Service
public class PedidoService {


    public void finaliza(Pedido pedido) {
        // efetua pagamento no gateway
        // dá baixa no estoque
        // atualiza pedido
        // envia email de confirmação
        // gera nota fiscal
    }
}
```

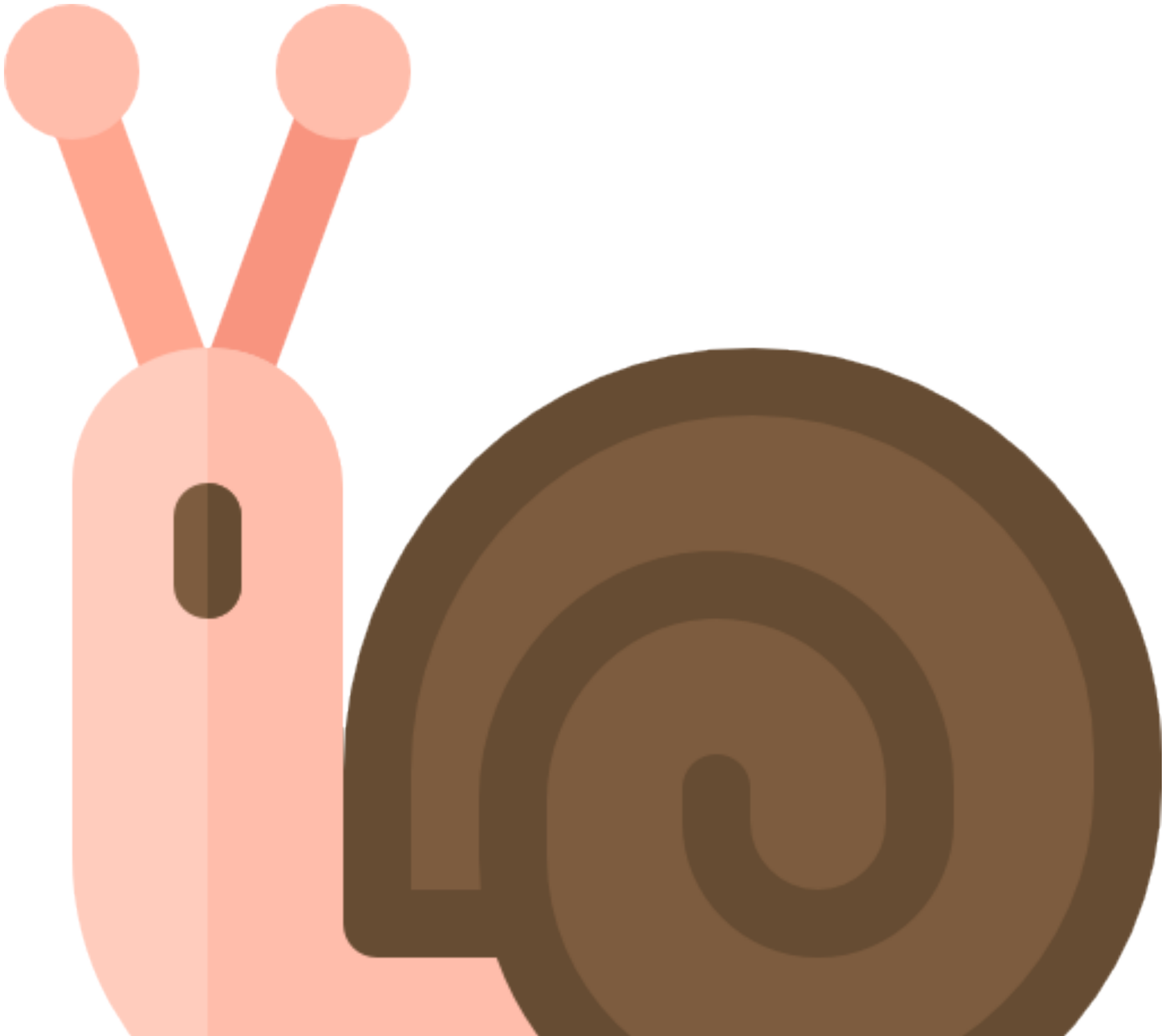
```java
@Service
public class PedidoService {


    public void finaliza(Pedido pedido) {
        // efetua pagamento no gateway
        // dá baixa no estoque
        // atualiza pedido
        // envia email de confirmação
        // gera nota fiscal
    }
}
```

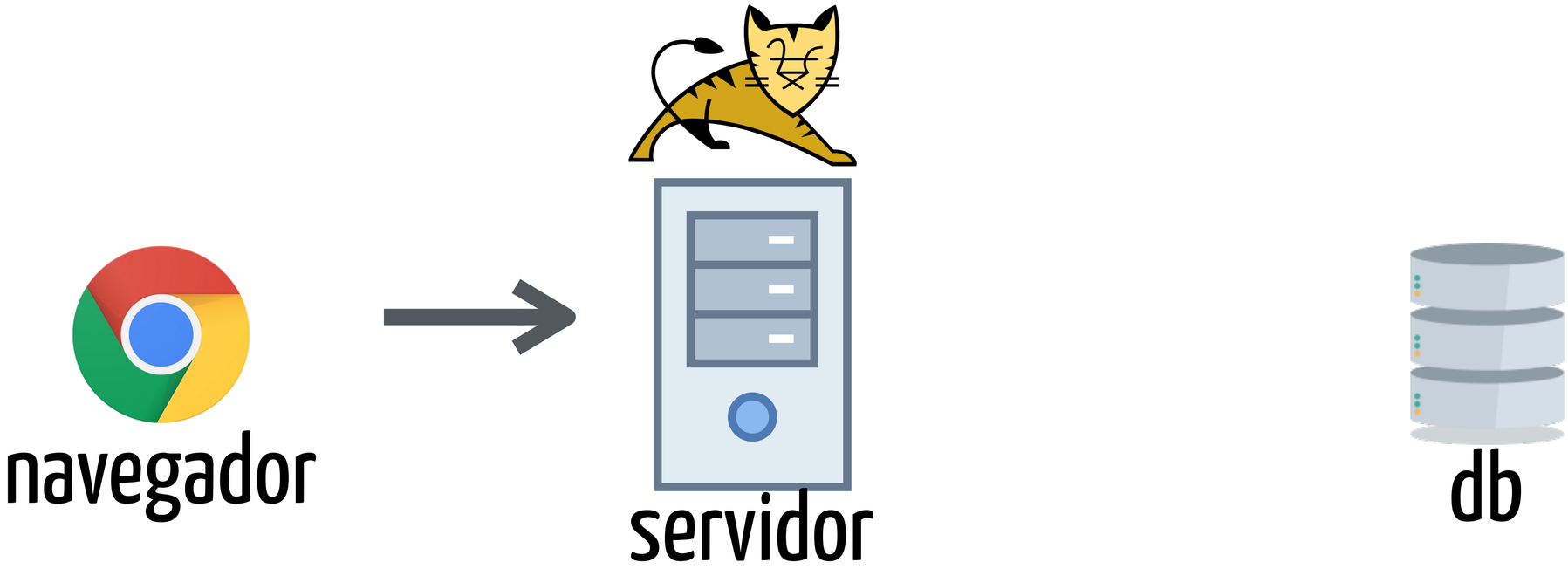
```java
@Service
public class PedidoService {


    public void finaliza(Pedido pedido) {
        // efetua pagamento no gateway
        // dá baixa no estoque
        // atualiza pedido
        // envia email de confirmação
        // gera nota fiscal
    }
}
```

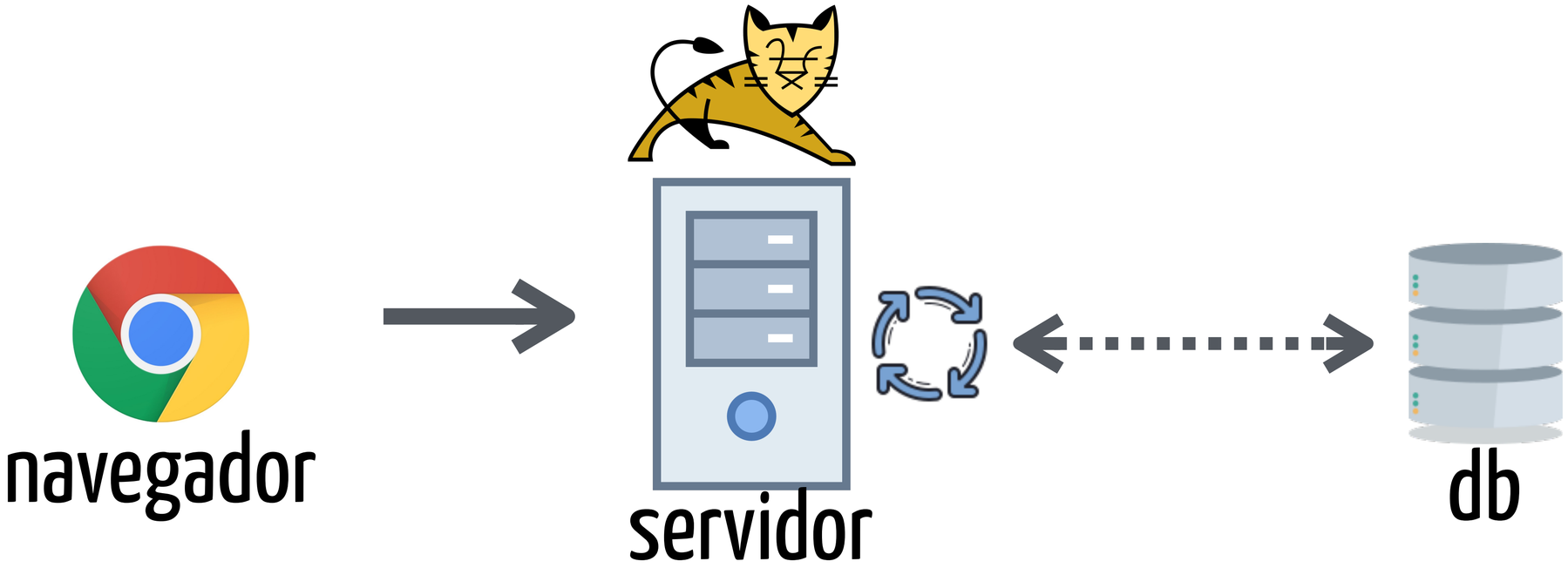
```java
@Service
public class PedidoService {


    public void finaliza(Pedido pedido) {
        // efetua pagamento no gateway
        // dá baixa no estoque
        // atualiza pedido
        // envia email de confirmação
        // gera nota fiscal
    }
}
```

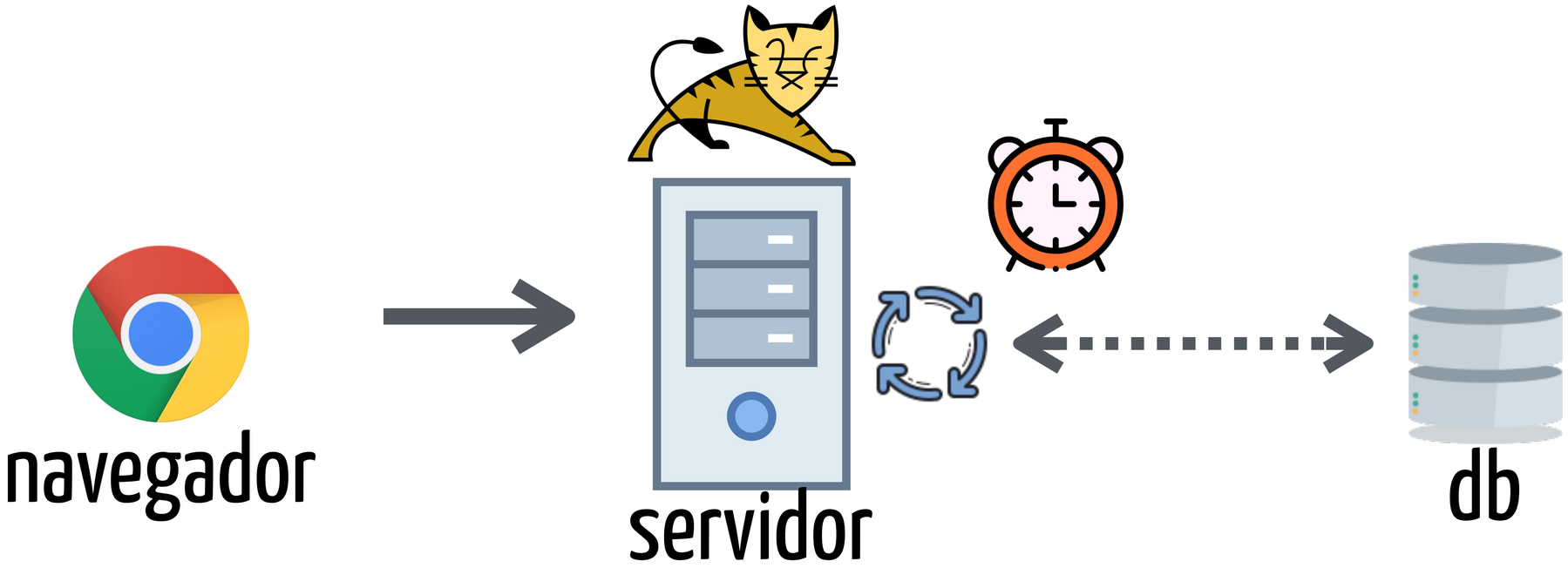
```java
@Service
public class PedidoService {


    public void finaliza(Pedido pedido) {
        // efetua pagamento no gateway
        // dá baixa no estoque
        // atualiza pedido
        // envia email de confirmação
        // gera nota fiscal
    }
}
```

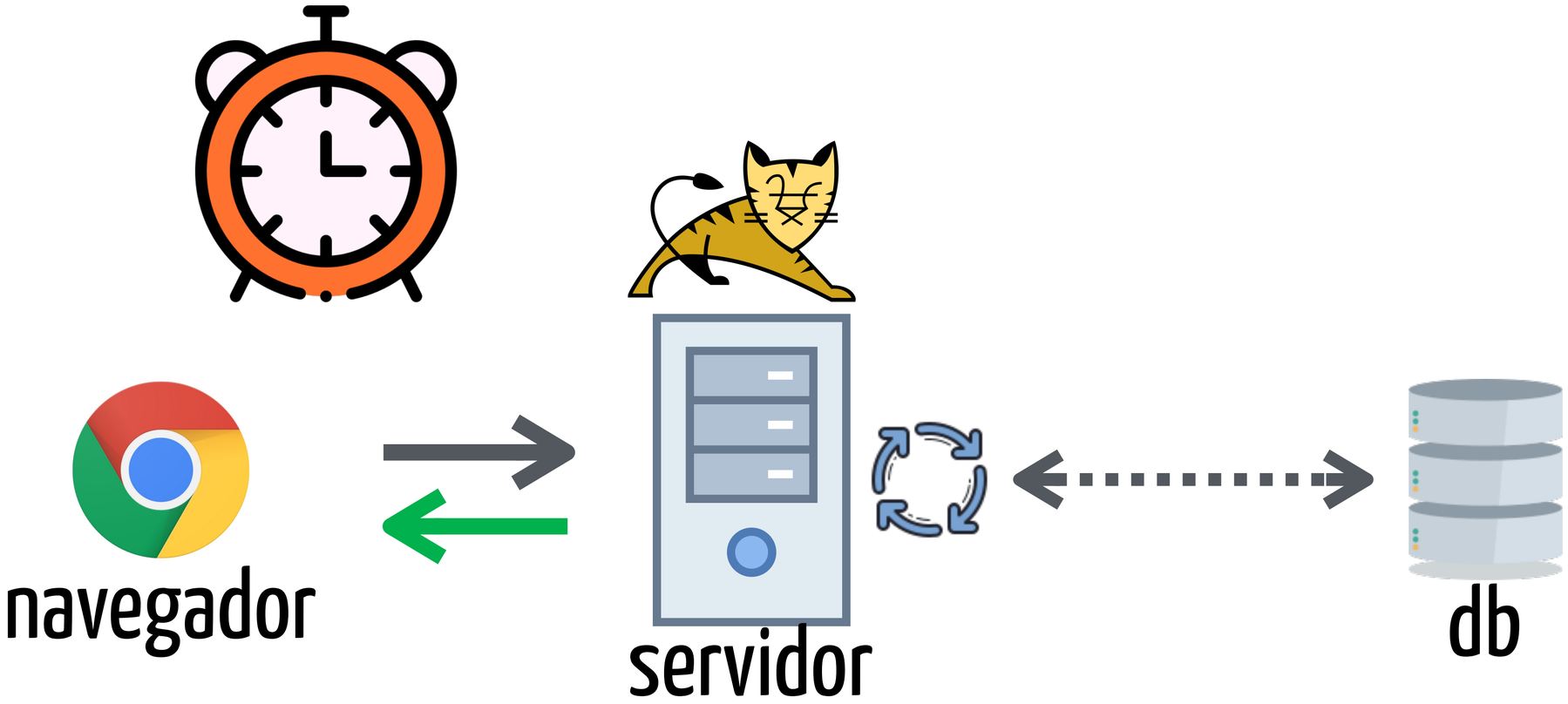
```java
@Service
public class PedidoService {


    public void finaliza(Pedido pedido) {
        // efetua pagamento no gateway
        // dá baixa no estoque
        // atualiza pedido
        // envia email de confirmação
        // gera nota fiscal
    }
}
```




navegador
servidor
db


navegador
servidor
db


navegador
servidor
db


navegador
servidor
db

se a execução é custosa então
ela é um possível **gargalo**...

ASSINCRONICIDADE
(processa mais tarde)

```java
@Service
public class PedidoService {


    public void finaliza(Pedido pedido) {
        // efetua pagamento no gateway
        // dá baixa no estoque
        // atualiza pedido
        // envia email de confirmação
        // gera nota fiscal
    }
}
```

```java
@Service
public class PedidoService {

    @Async
    public void finaliza(Pedido pedido) {
        // efetua pagamento no gateway
        // dá baixa no estoque
        // atualiza pedido
        // envia email de confirmação
        // gera nota fiscal
    }
}
```

Pool de Threads

```java
@Controller
public class PedidoController {

    @Autowired
    public PedidoService service;

    @PostMapping(path="/vendas/finaliza", ...)
    public void finalizaPedido(Pedido pedido) {
      // faz alguma validação de dados

      service.finaliza(pedido);
    }
}
```

```java
@Controller
public class PedidoController {

    @Autowired
    public PedidoService service;

    @PostMapping(path="/vendas/finaliza", ...)
    public void finalizaPedido(Pedido pedido) {
      // faz alguma validação de dados

      service.finaliza(pedido);
    }
}
```

IMEDIATO!

O que acontece por baixo do panos?

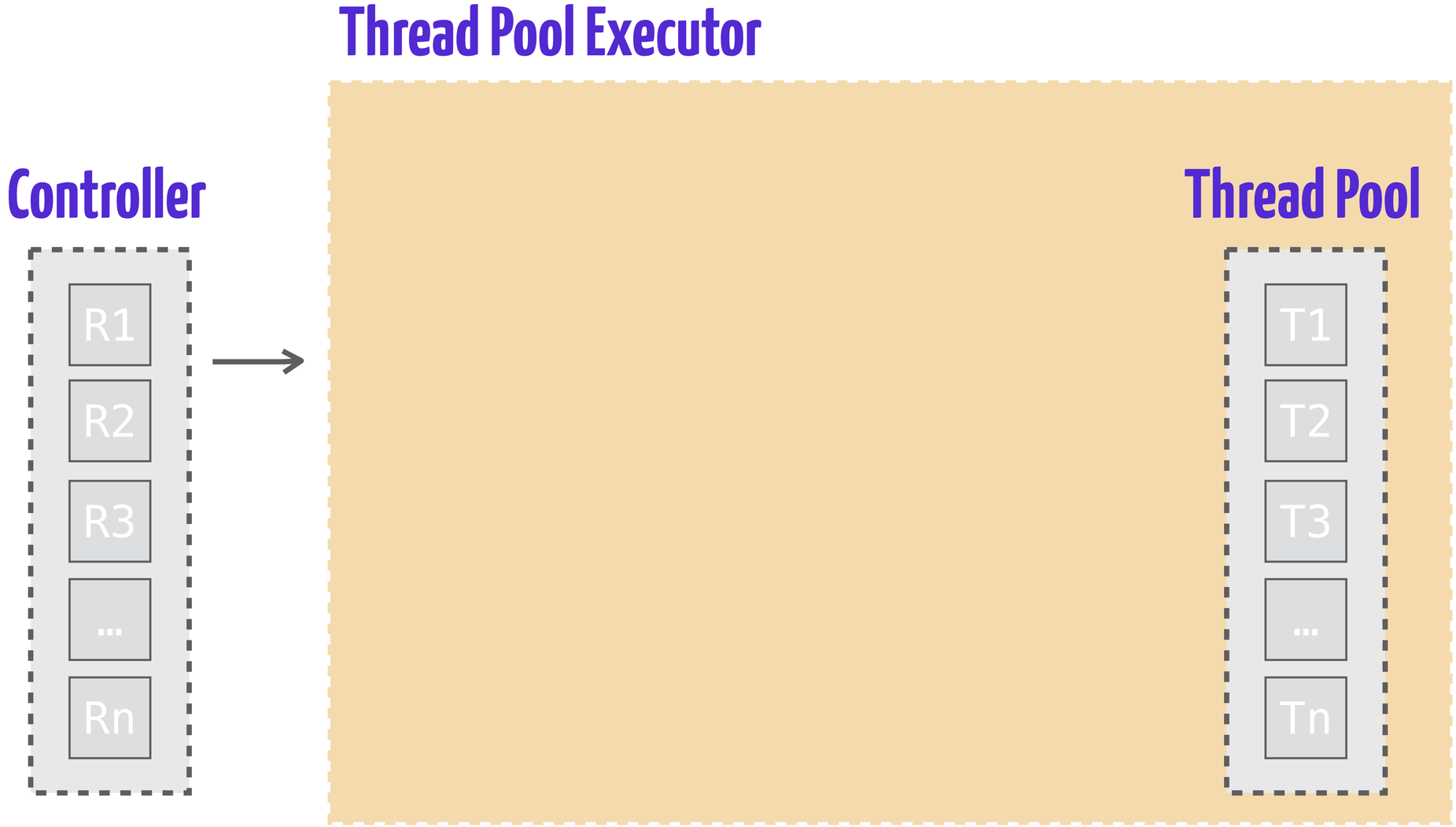

Controller
R1
R2
R3
...
Rn
service.finaliza(pedido);
Thread Pool

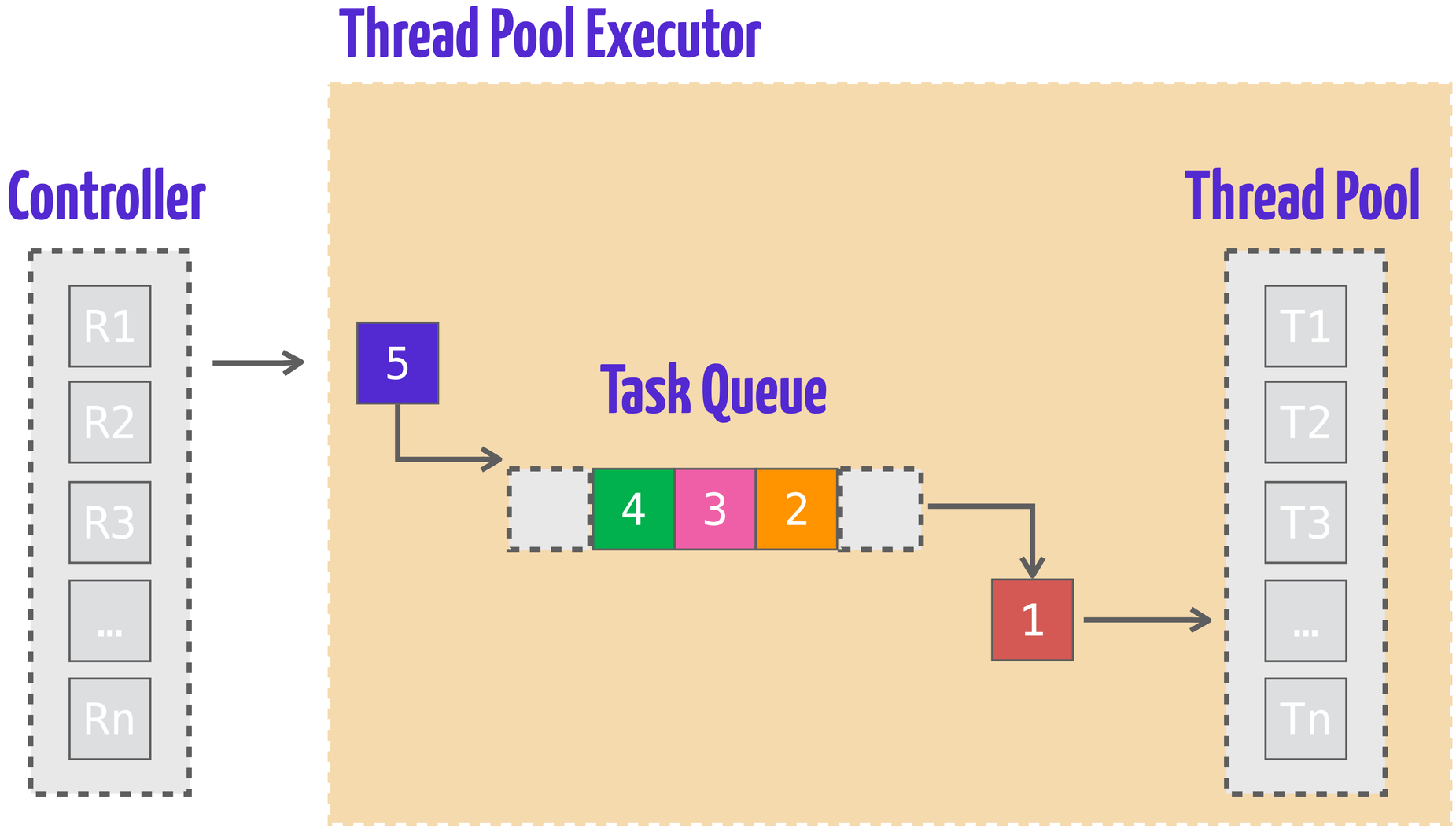

Controller
R1
R2
R3
...
Rn
new Thread(() -> {
service.finaliza(pedido);
}).start();
Thread Pool

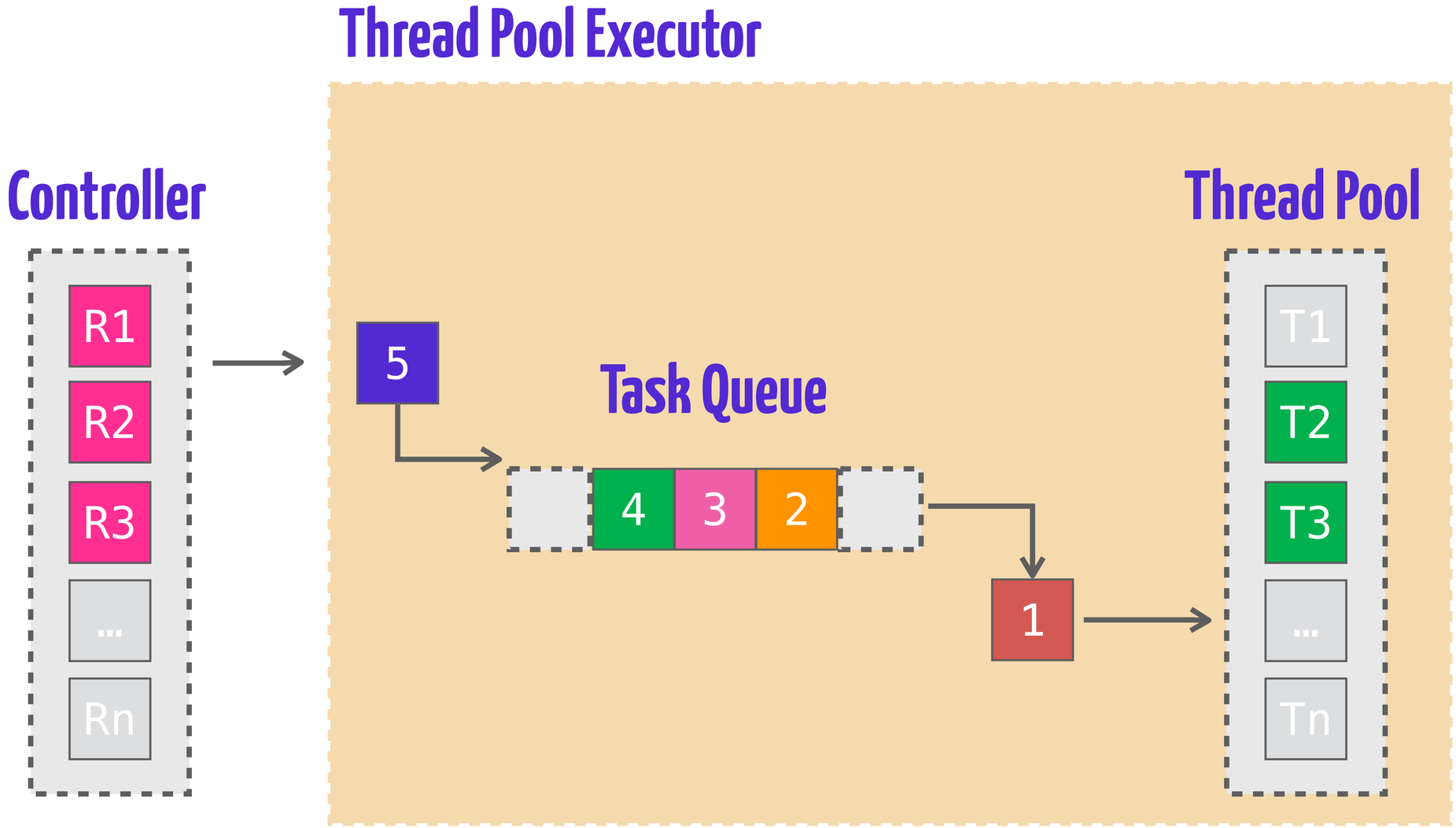

Controller
R1
R2
R3
...
Rn
new Thread(() -> {
service.finaliza(pedido);
}).start();
Thread Pool
T1

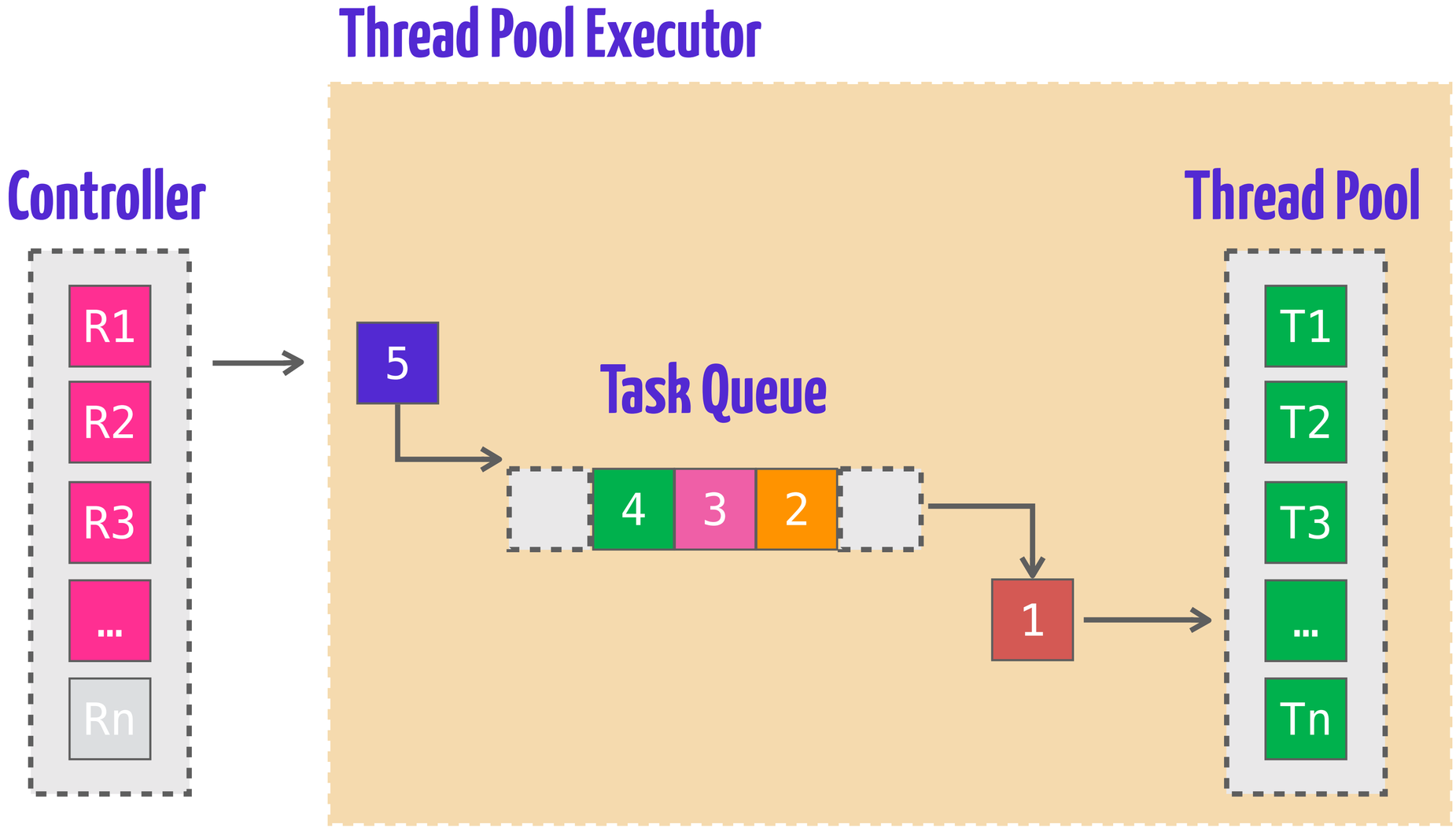

Controller
R1
R2
R3
...
Rn
new Thread(() -> {
service.finaliza(pedido);
}).start();
Thread Pool
T1
T2

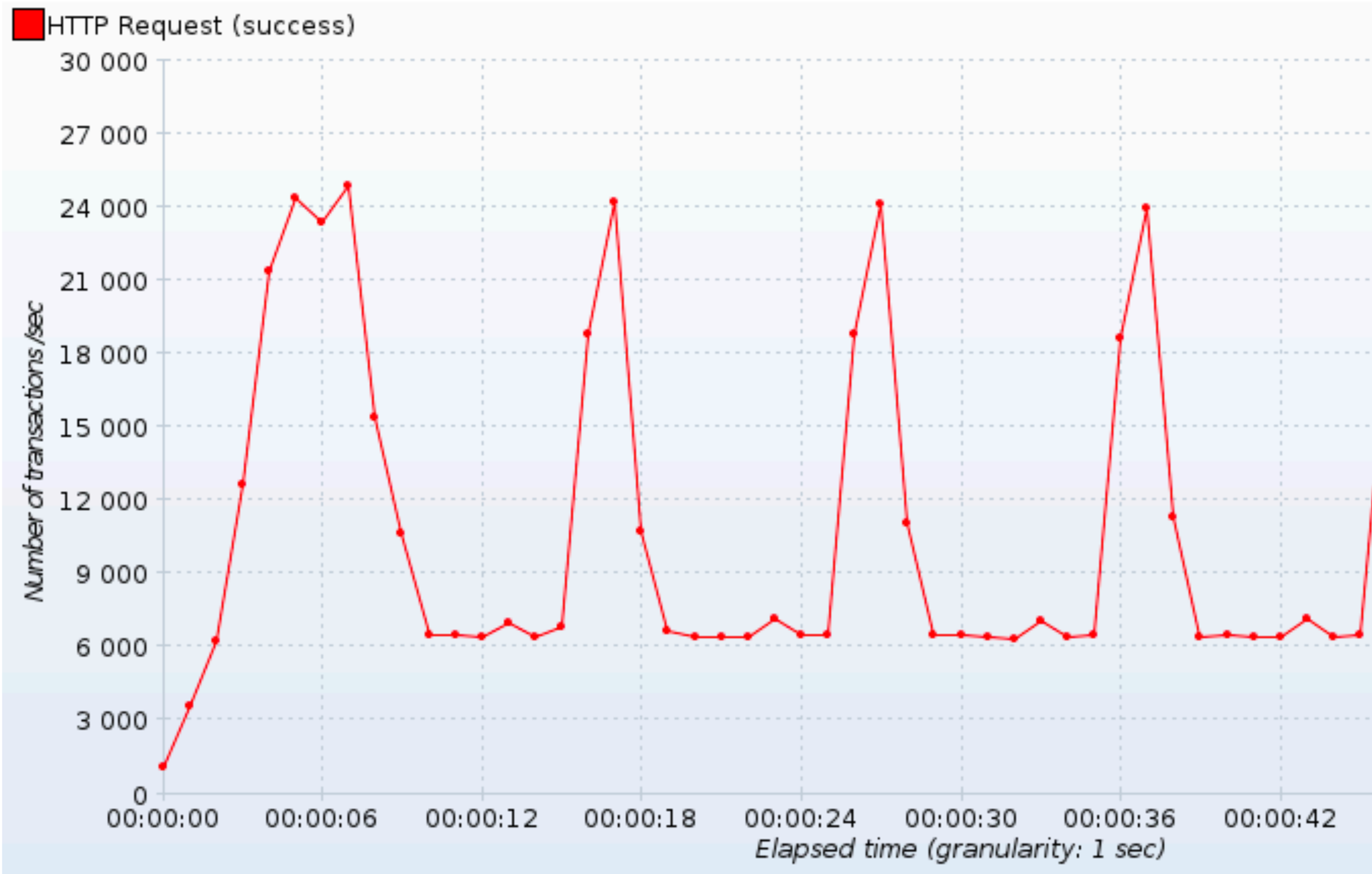
É um pouco mais engenhoso do que isso...

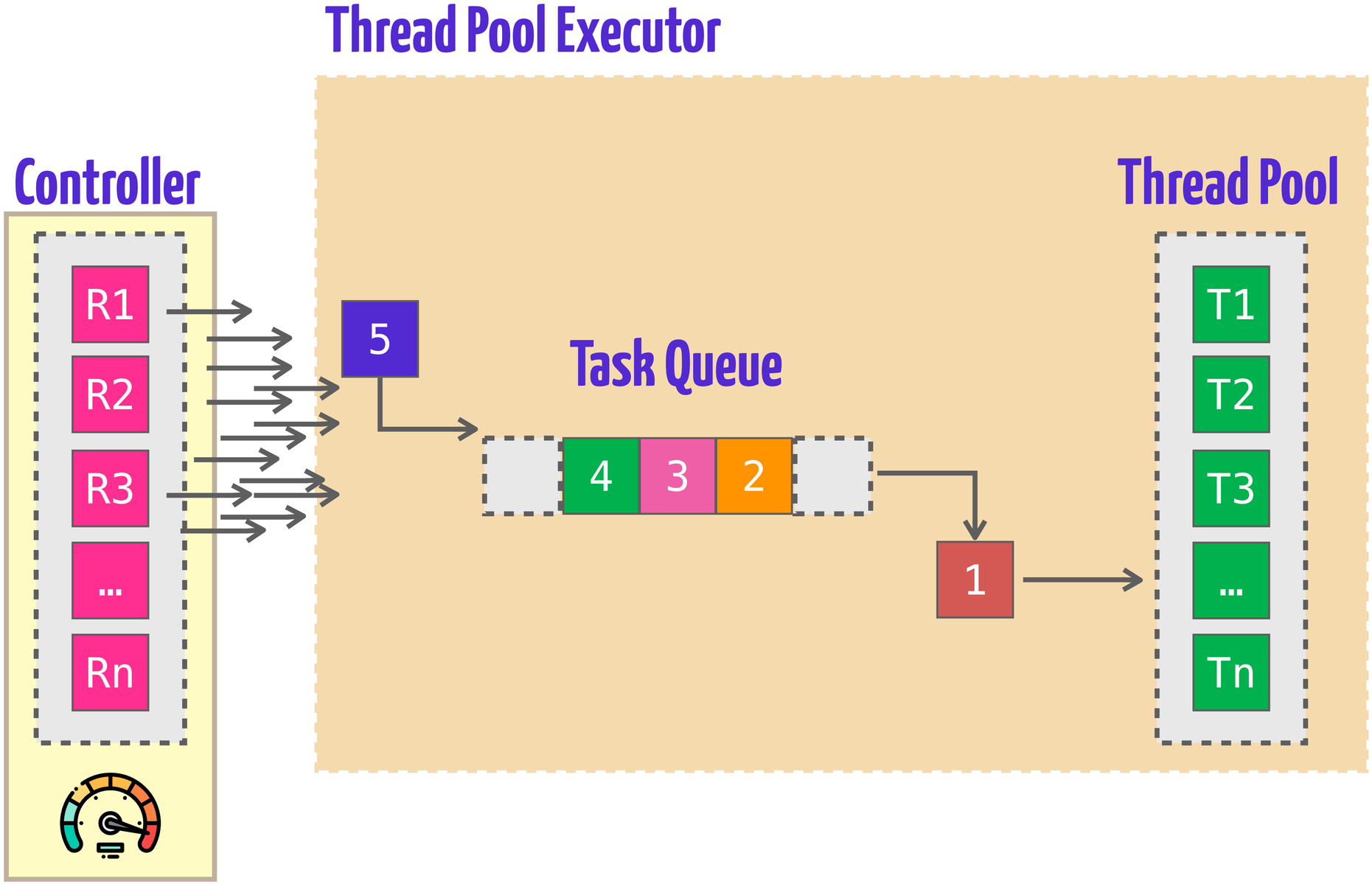

Thread Pool Executor
Controller
R1
R2
R3
...
Rn
Thread Pool
T1
T2
T3
...
Tn

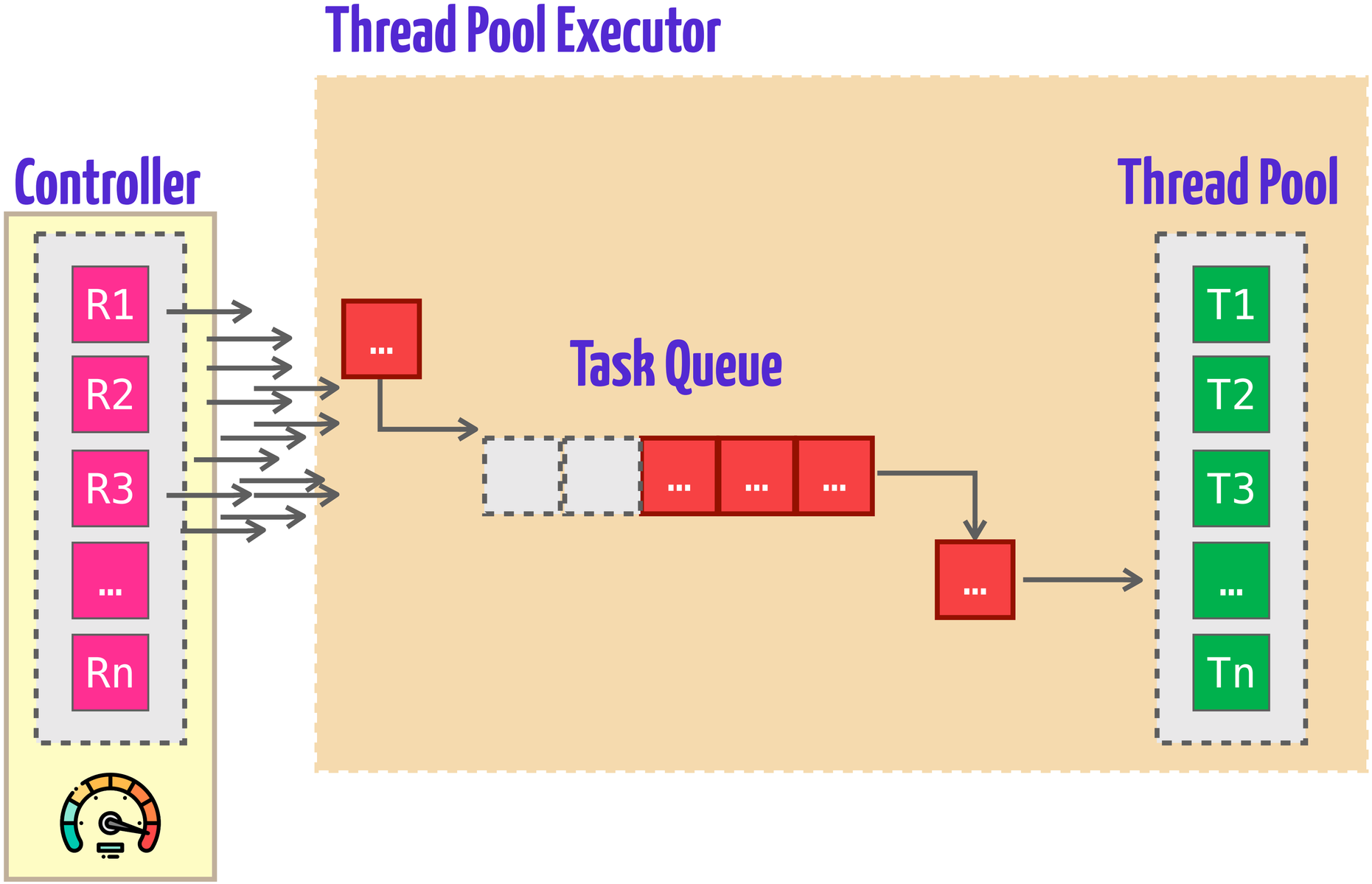

Thread Pool Executor
Controller
R1
R2
R3
...
Rn
5
Task Queue
4
3
2
1
Thread Pool
T1
T2
T3
...
Tn


Thread Pool Executor
Controller
R1
R2
R3
...
Rn
5
Task Queue
4
3
2
1
Thread Pool
T1
T2
T3
...
Tn

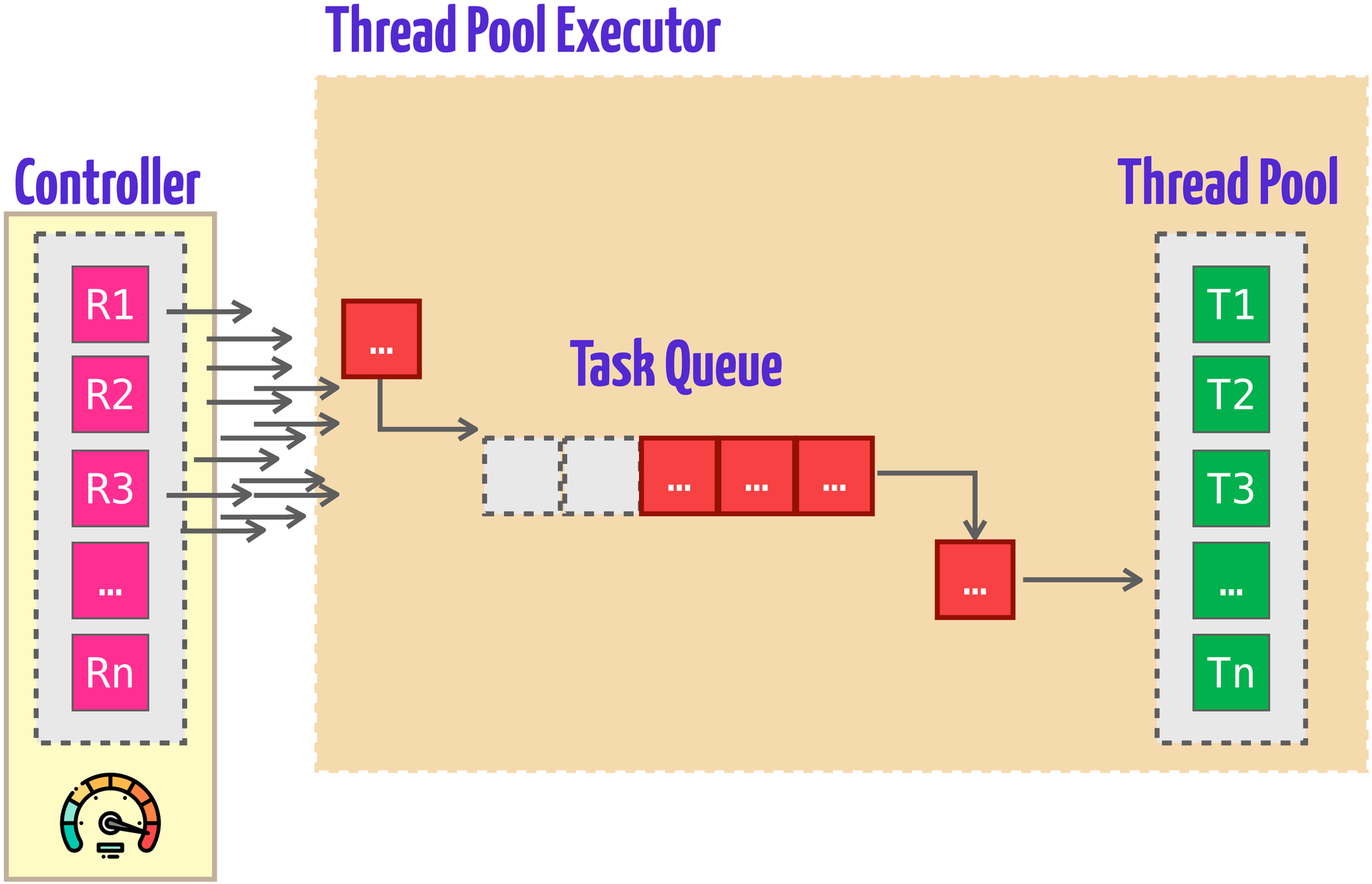

Thread Pool Executor
Controller
R1
R2
R3
...
Rn
5
Task Queue
4
3
2
1
Thread Pool
T1
T2
T3
...
Tn

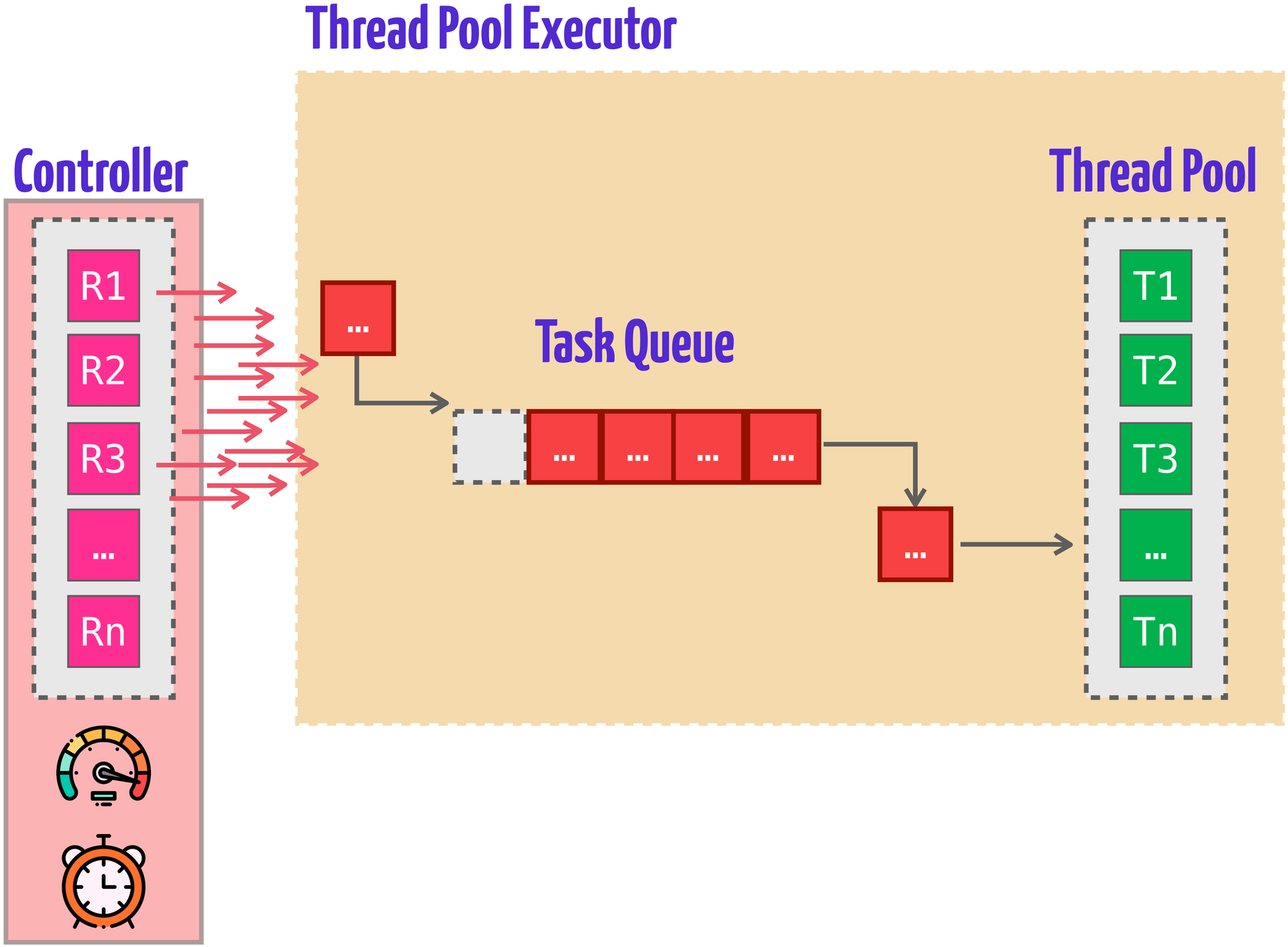

HTTP Request (success)
30 000
27 000
24 000
21 000
18 000
15 000
12 000
9 000
6 000
3 000
0
Number of transactions /sec
00:00:00
00:00:06
00:00:12
00:00:18
00:00:24
00:00:30
00:00:36
00:00:42
Elapsed time (granularity: 1 sec)

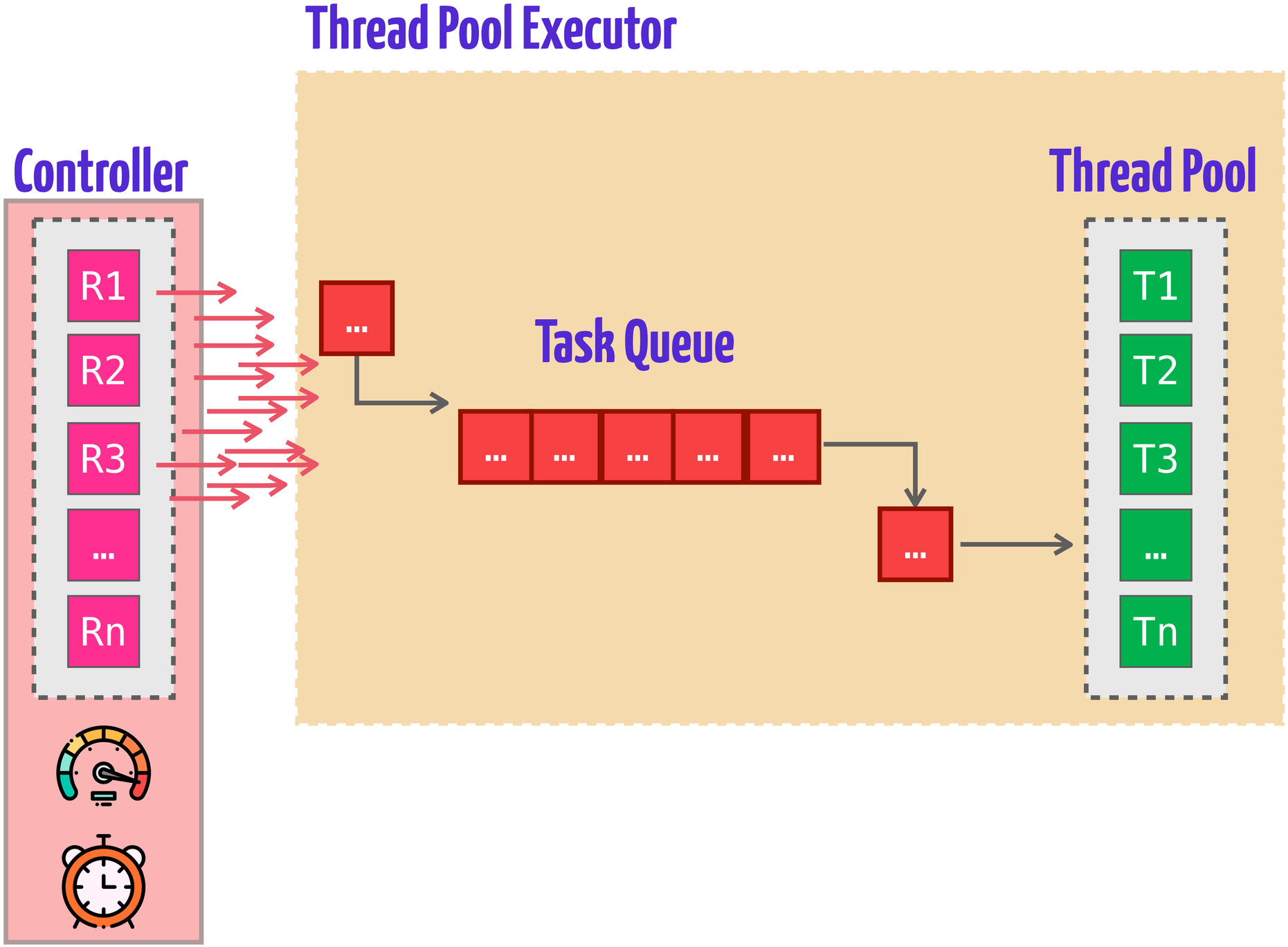

Thread Pool Executor
Controller
R1
R2
R3
...
Rn
5
Task Queue
4
3
2
1
Thread Pool
T1
T2
T3
...
Tn

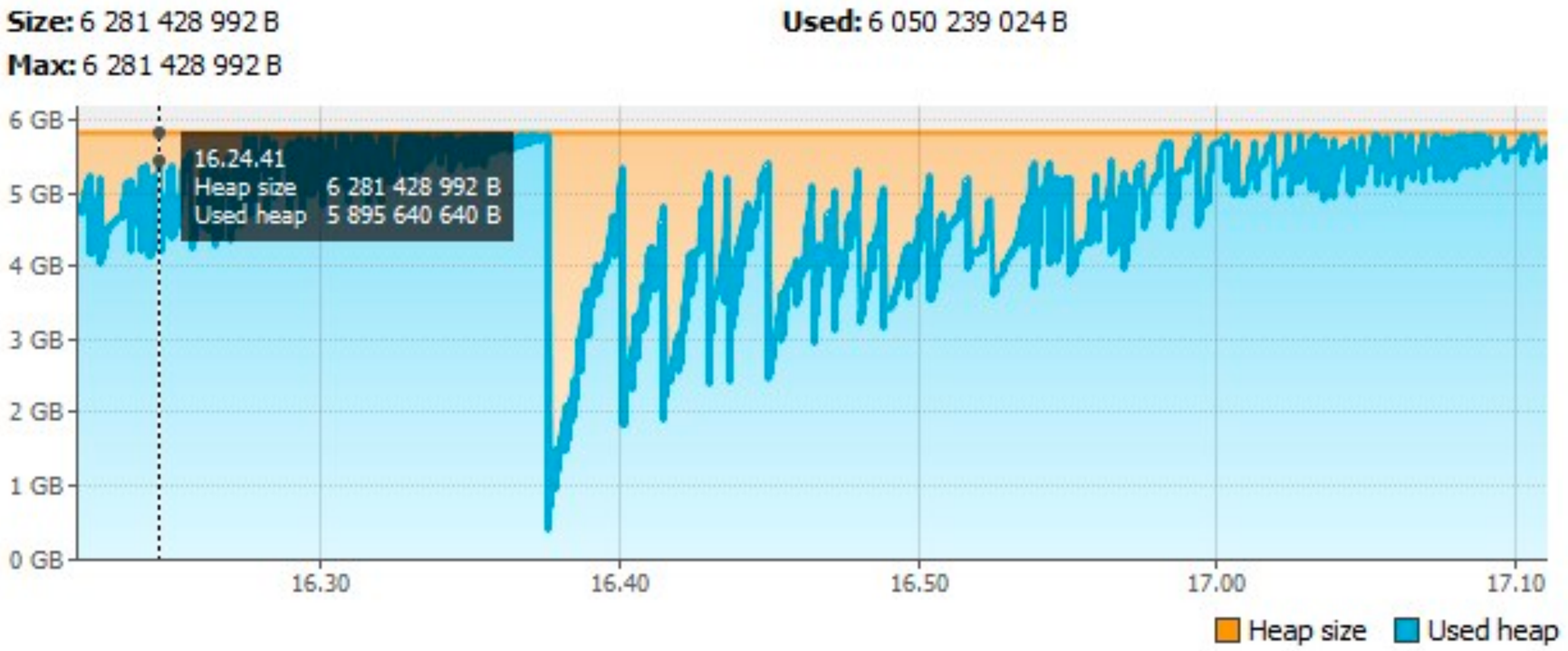

Thread Pool Executor
Controller
R1
R2
R3
...
Rn
...
Task Queue
...
...
...
...
Thread Pool
T1
T2
T3
...
Tn

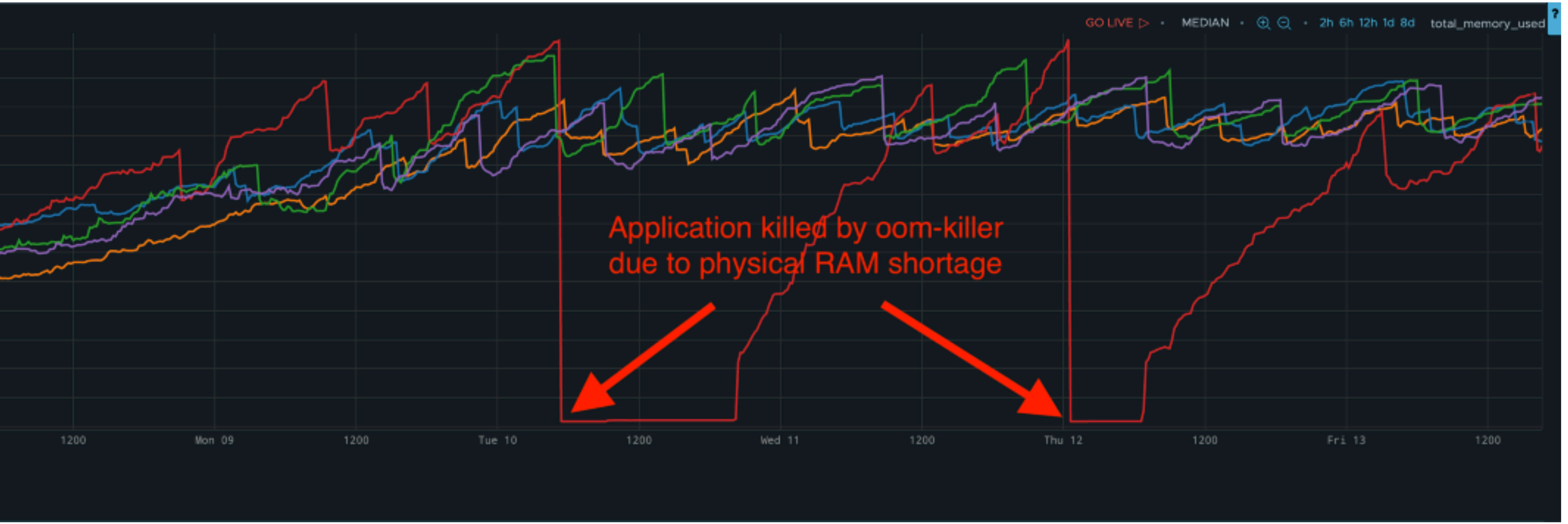
...e se além de
custosa ela também
for em **grande**
quantidade e longa?


Thread Pool Executor
Controller
R1
R2
R3
...
Rn
...
Task Queue
...
...
...
...
Thread Pool
T1
T2
T3
...
Tn


Thread Pool Executor
Controller
R1
R2
R3
...
Rn
...
Task Queue
...
...
...
...
...
Thread Pool
T1
T2
T3
...
Tn


Thread Pool Executor
Controller
R1
R2
R3
...
Rn
...
Task Queue
...
...
Thread Pool
T1
T2
T3
...
Tn


Size: 6 281 428 992 B
Used: 6 050 239 024 B
Max: 6 281 428 992 B
6 GB
5 GB
4 GB
3 GB
2 GB
1 GB
0 GB
16.24.41
Heap size 6 281 428 992 B
Used heap 5 895 640 640 B
16.30
16.40
16.50
17.00
17.10
Heap size
Used heap


GO LIVE
MEDIAN
2h 6h 12h 1d 8d
total_memory_used
Application killed by oom-killer
due to physical RAM shortage
1200
Mon 09
1200
Tue 10
1200
Wed 11
1200
Thu 12
1200
Fri 13
1200

E o que acontece com os itens na fila do pool de threads?

Precisamos de uma fila
ainda maior...

Precisamos de uma fila
ainda maior...

...mas também
que seja durável!

No mundo enterprise...

BROKER

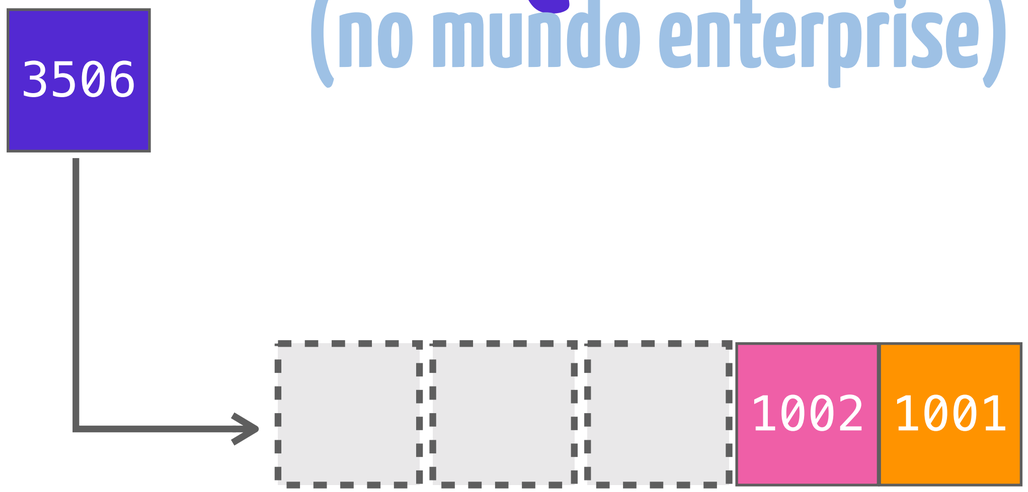
QUEUE
(no mundo enterprise)

QUEUE
(no mundo enterprise)

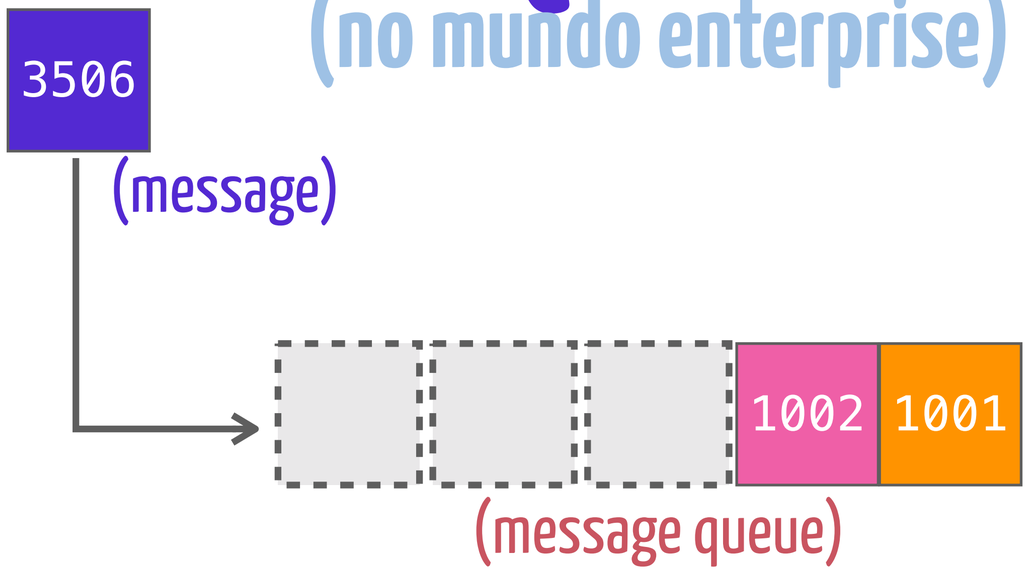
QUEUE
(no mundo enterprise)

(message queue)

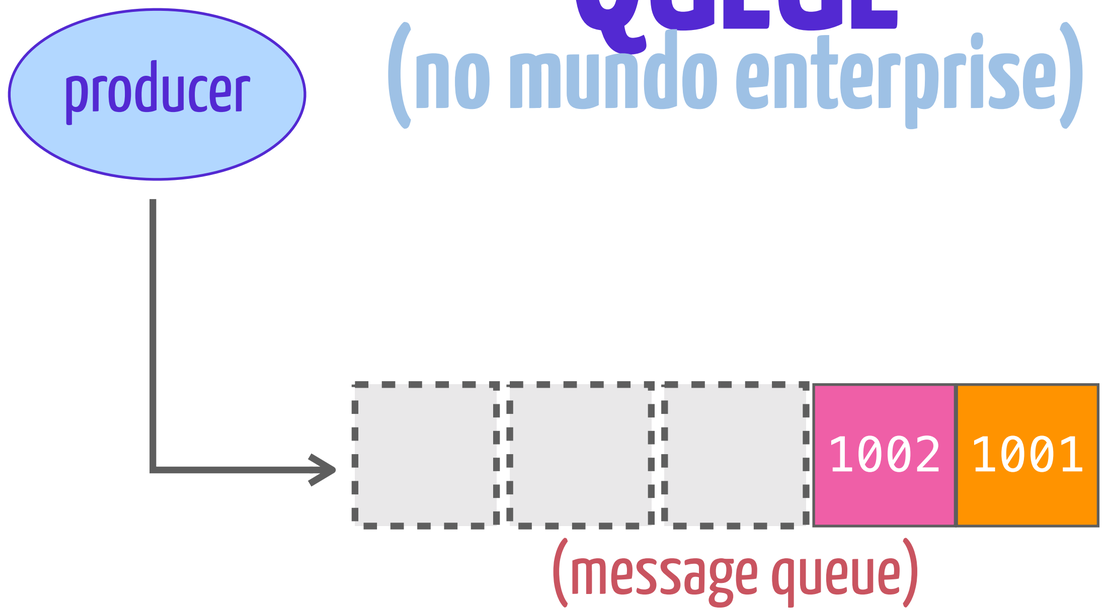
QUEUE
(no mundo enterprise)



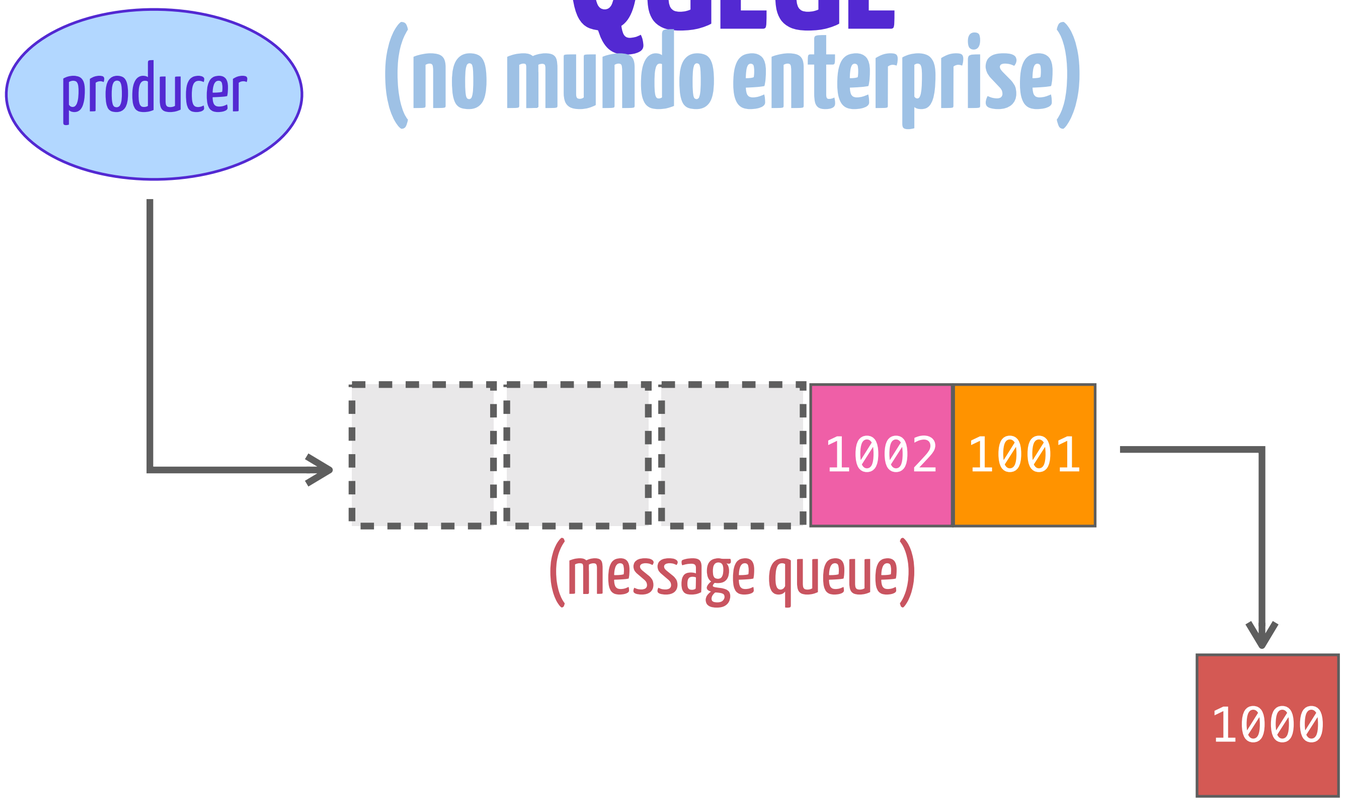
QUEUE
(no mundo enterprise)



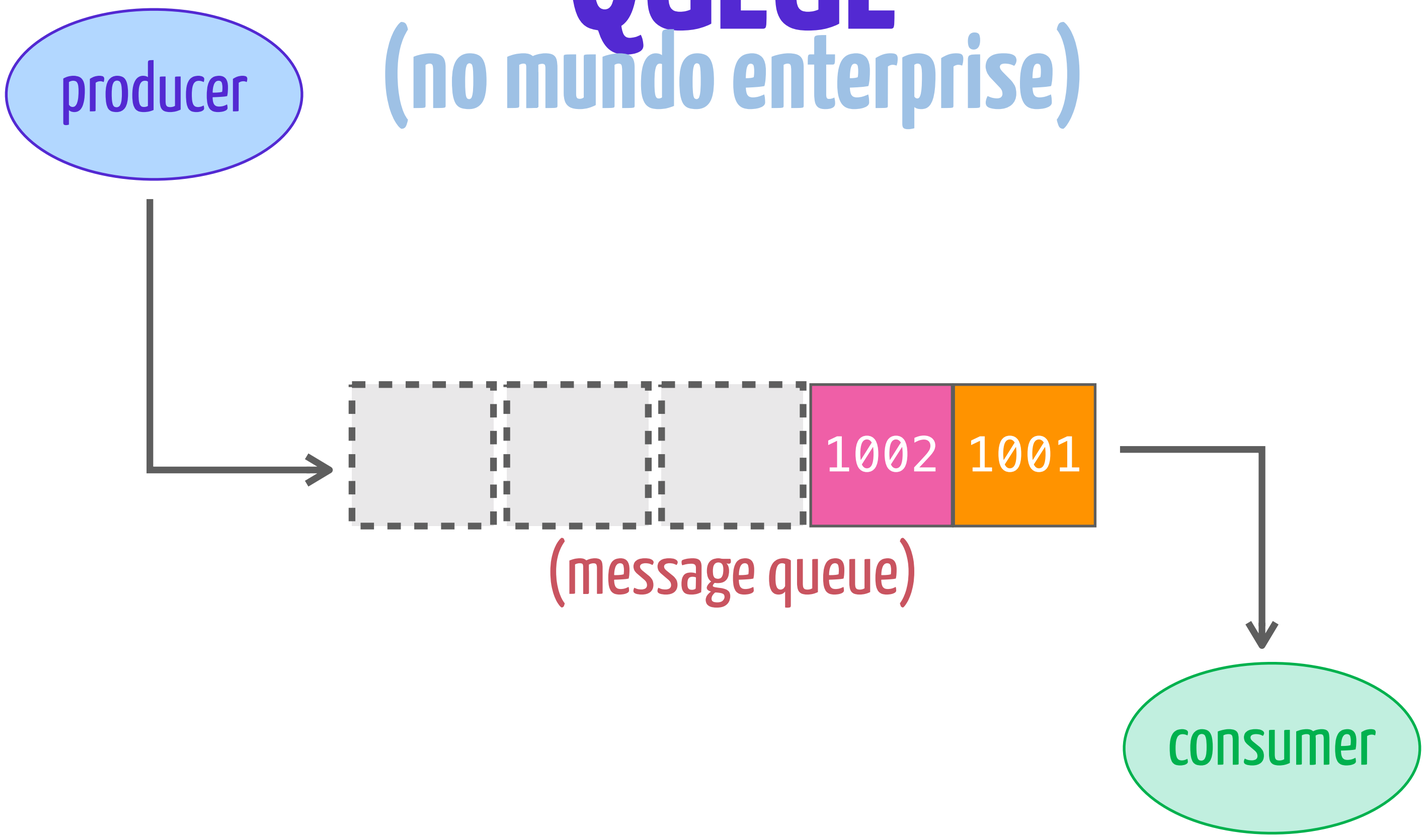
QUEUE
(no mundo enterprise)



QUEUE
(no mundo enterprise)



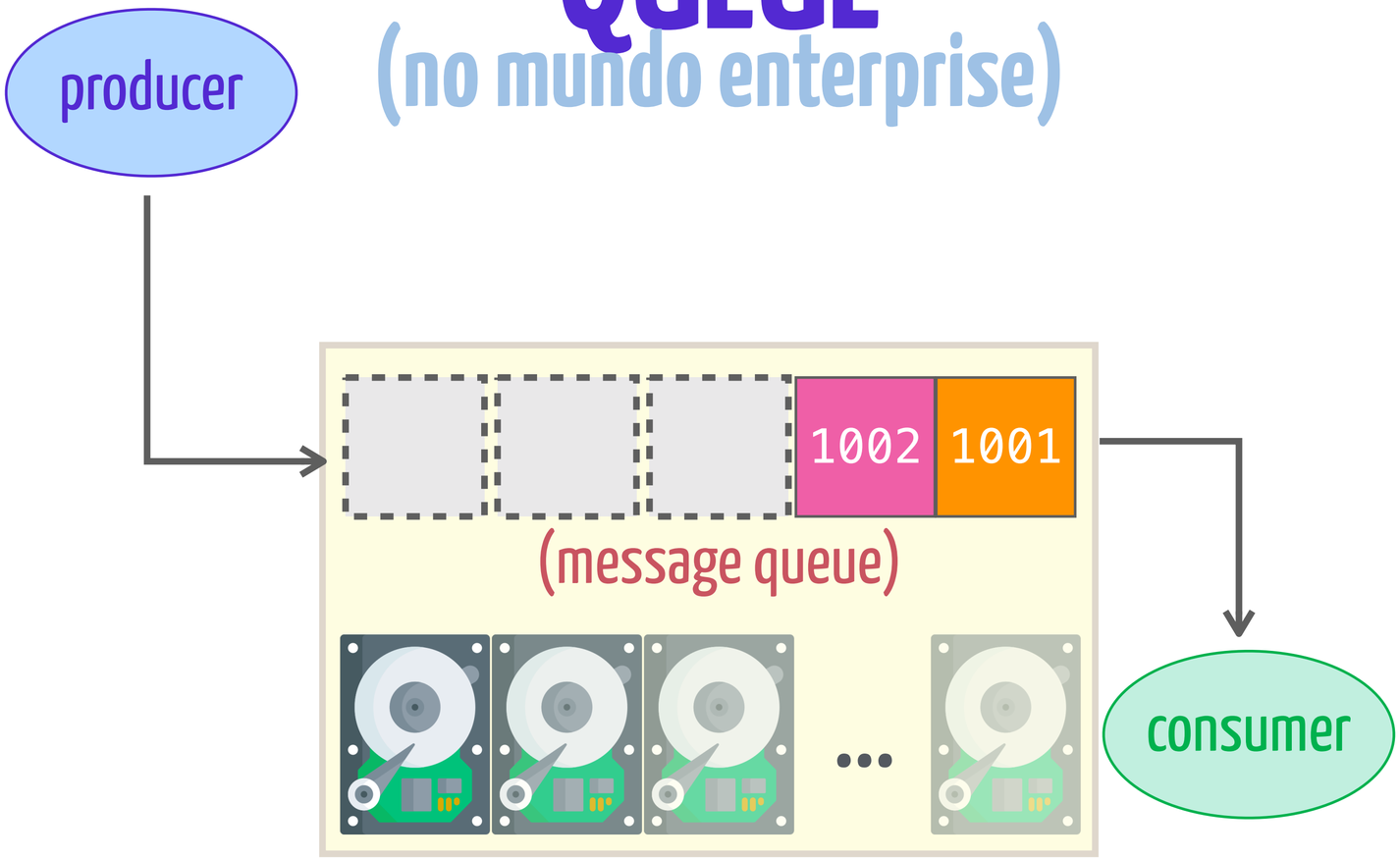
QUEUE
(no mundo enterprise)



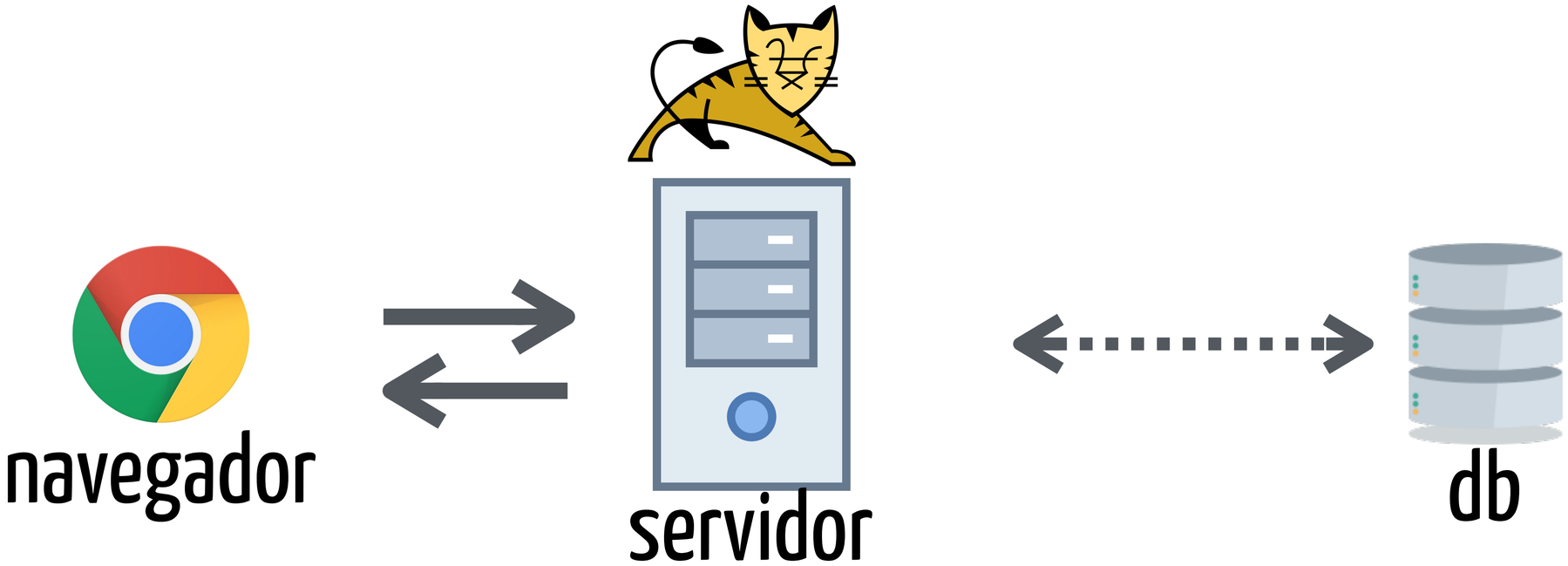
QUEUE
(no mundo enterprise)

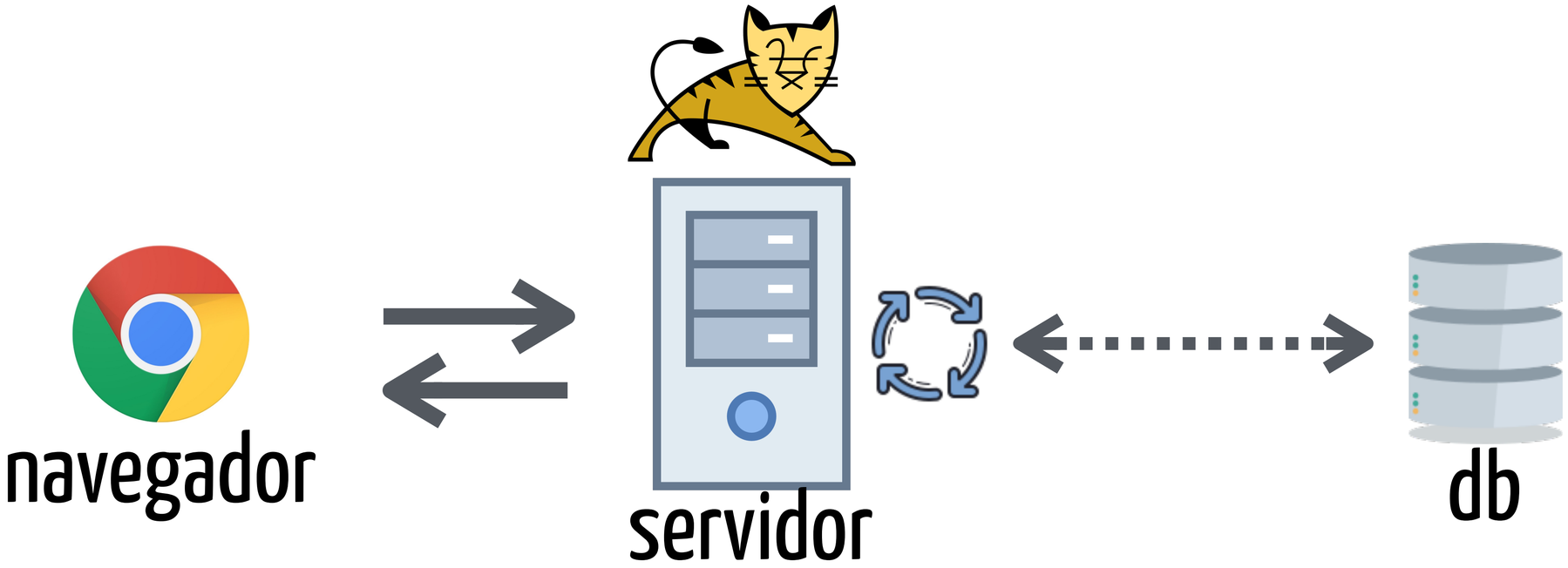
QUEUE

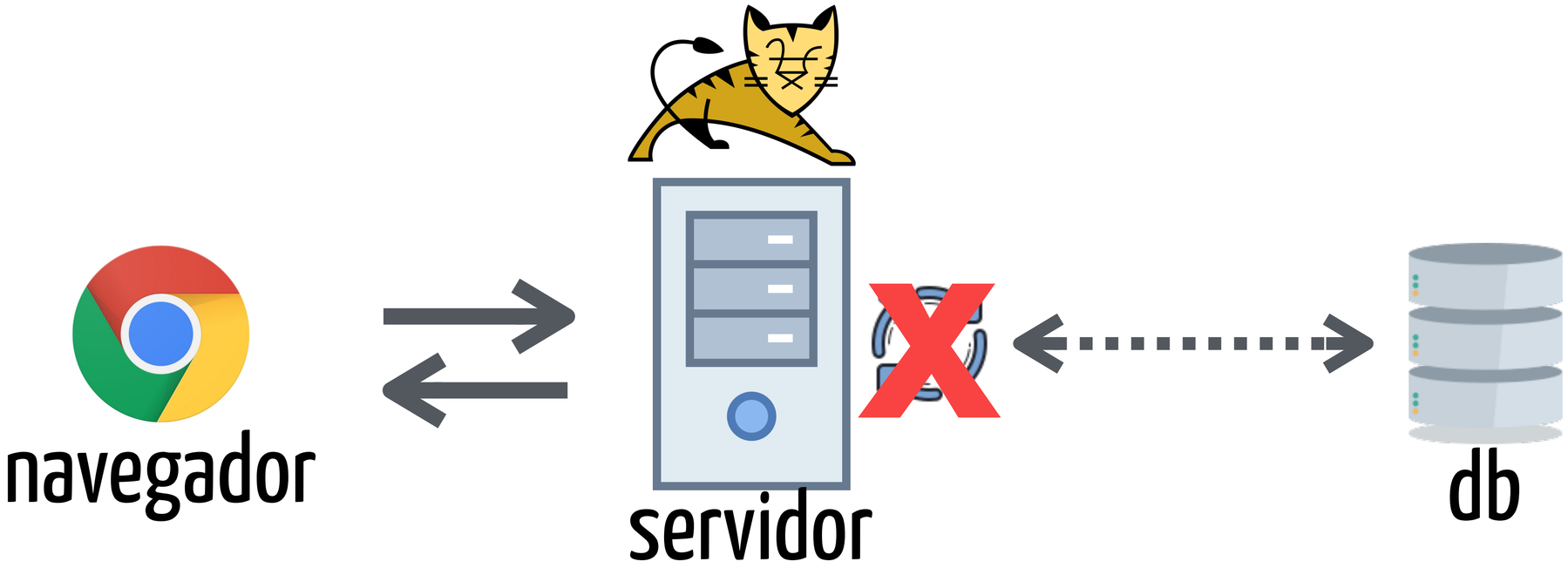
(no mundo enterprise)

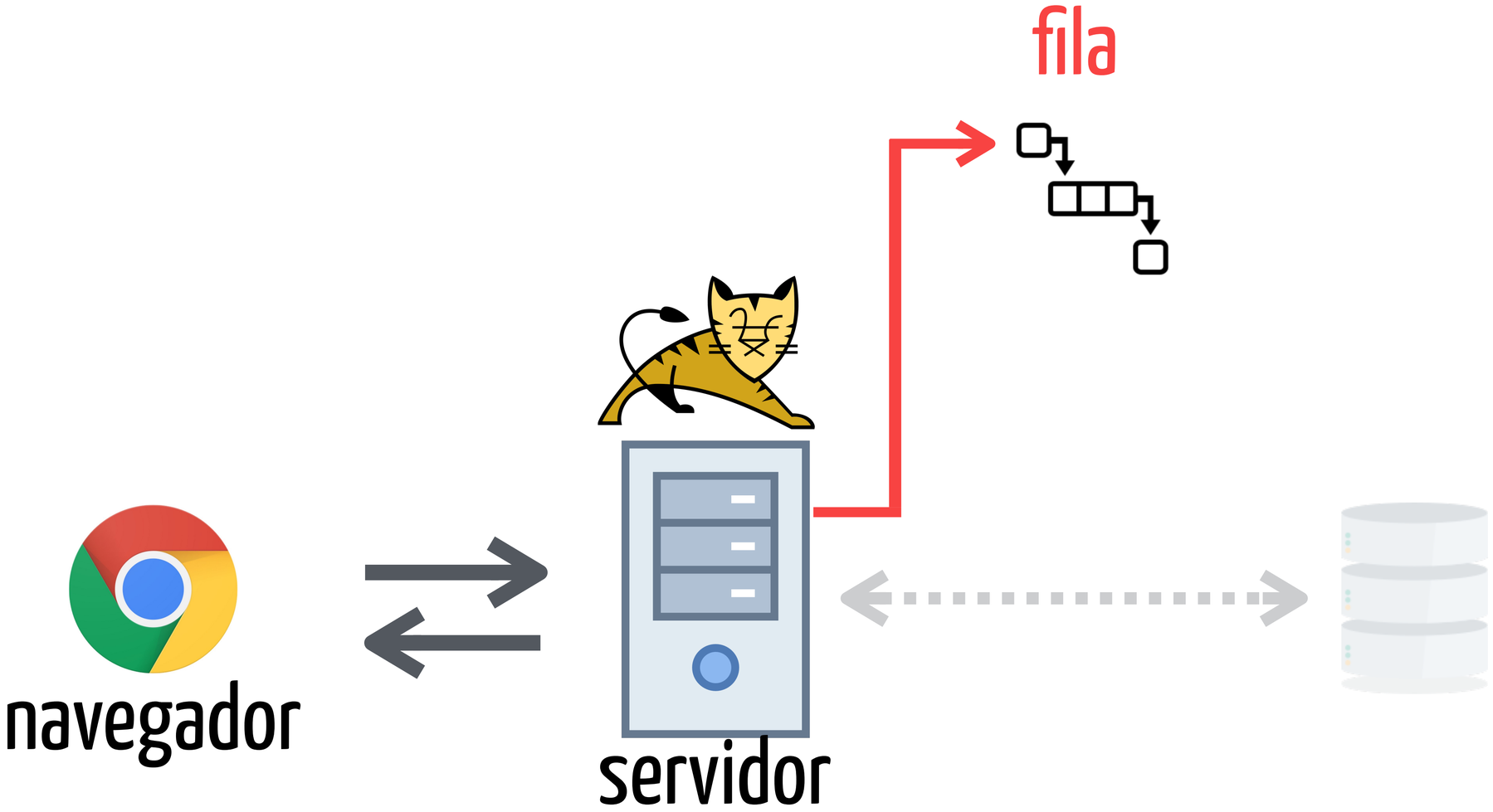


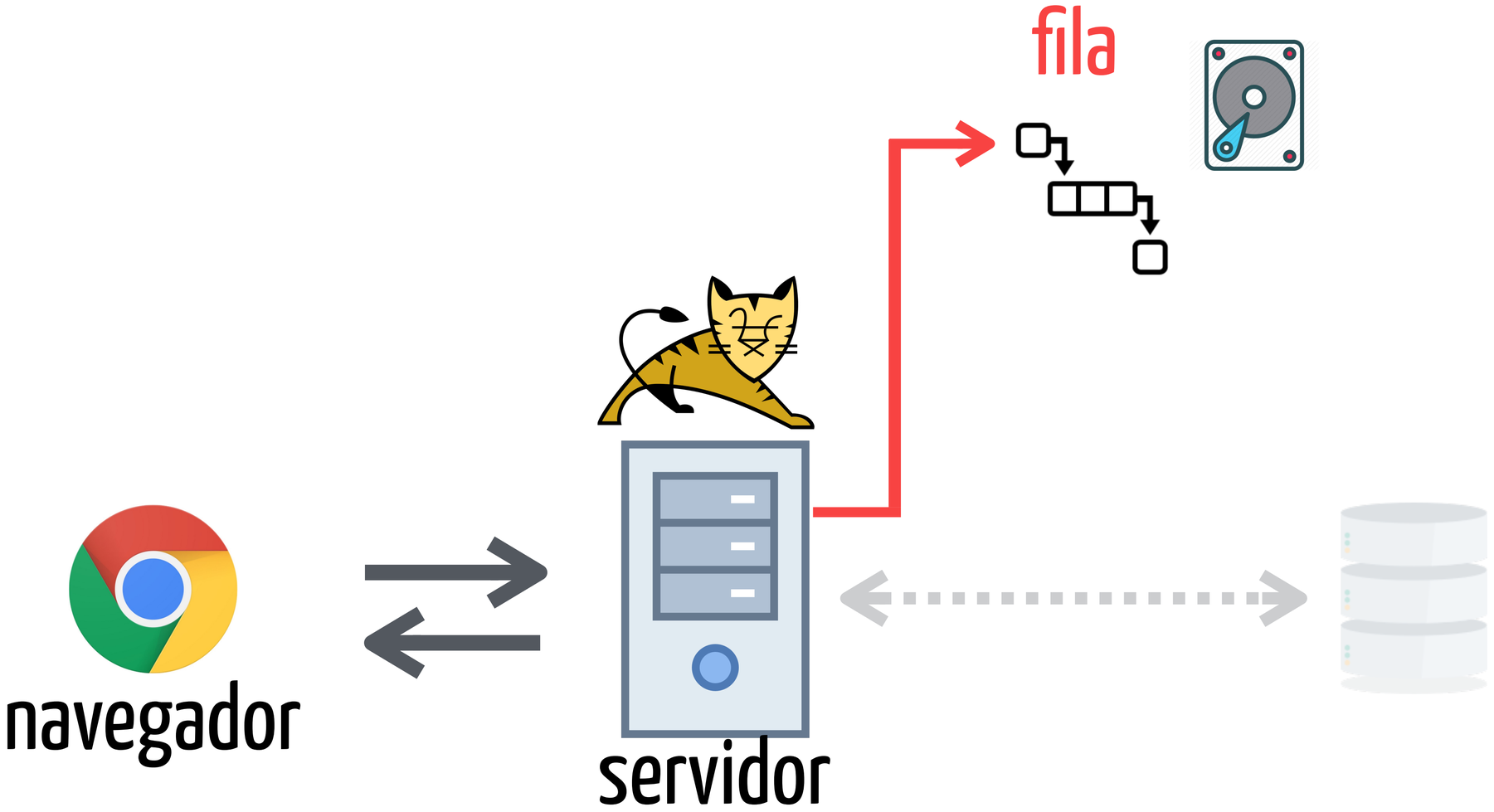
Ou seja...

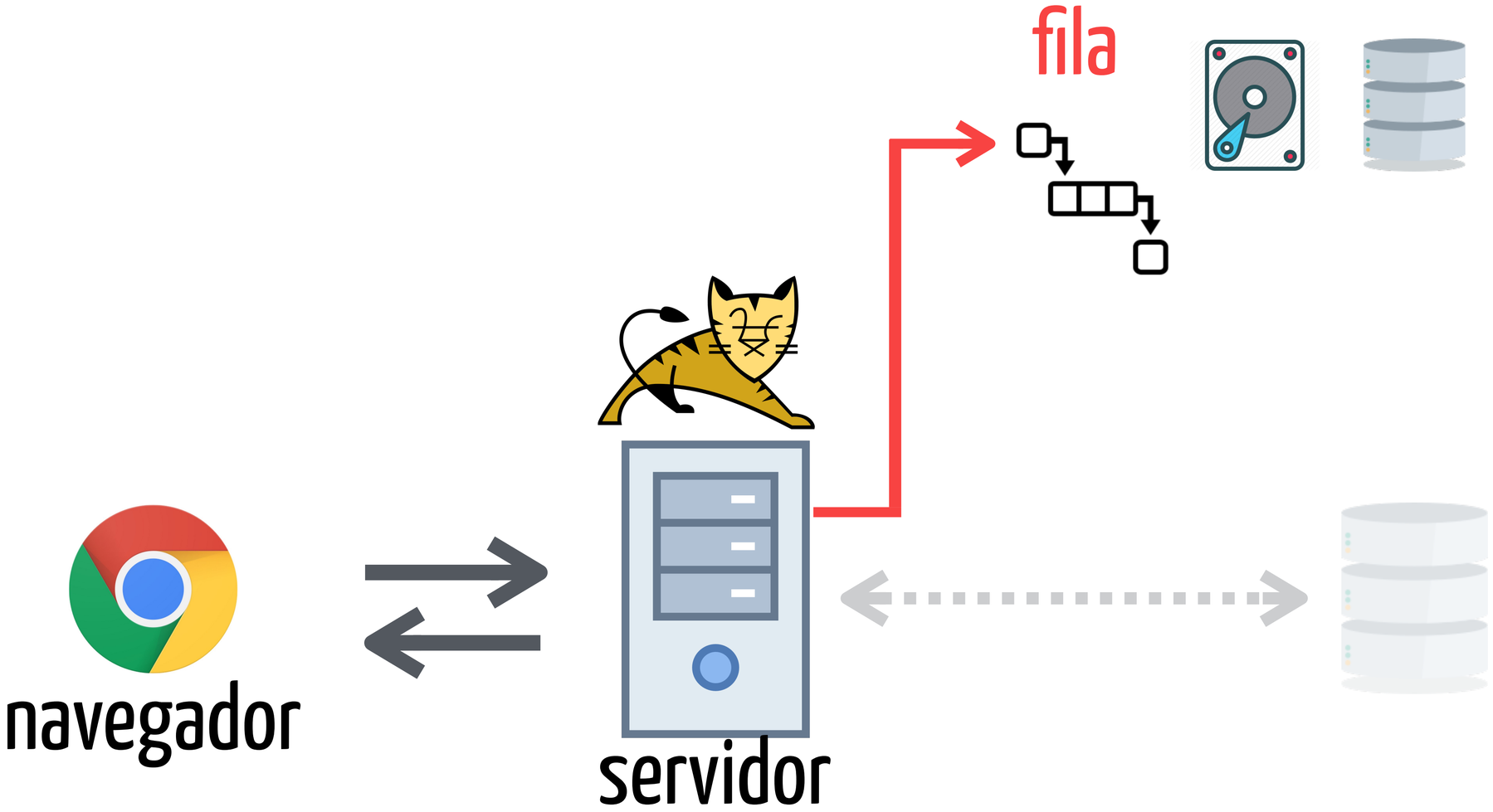

navegador
servidor
db

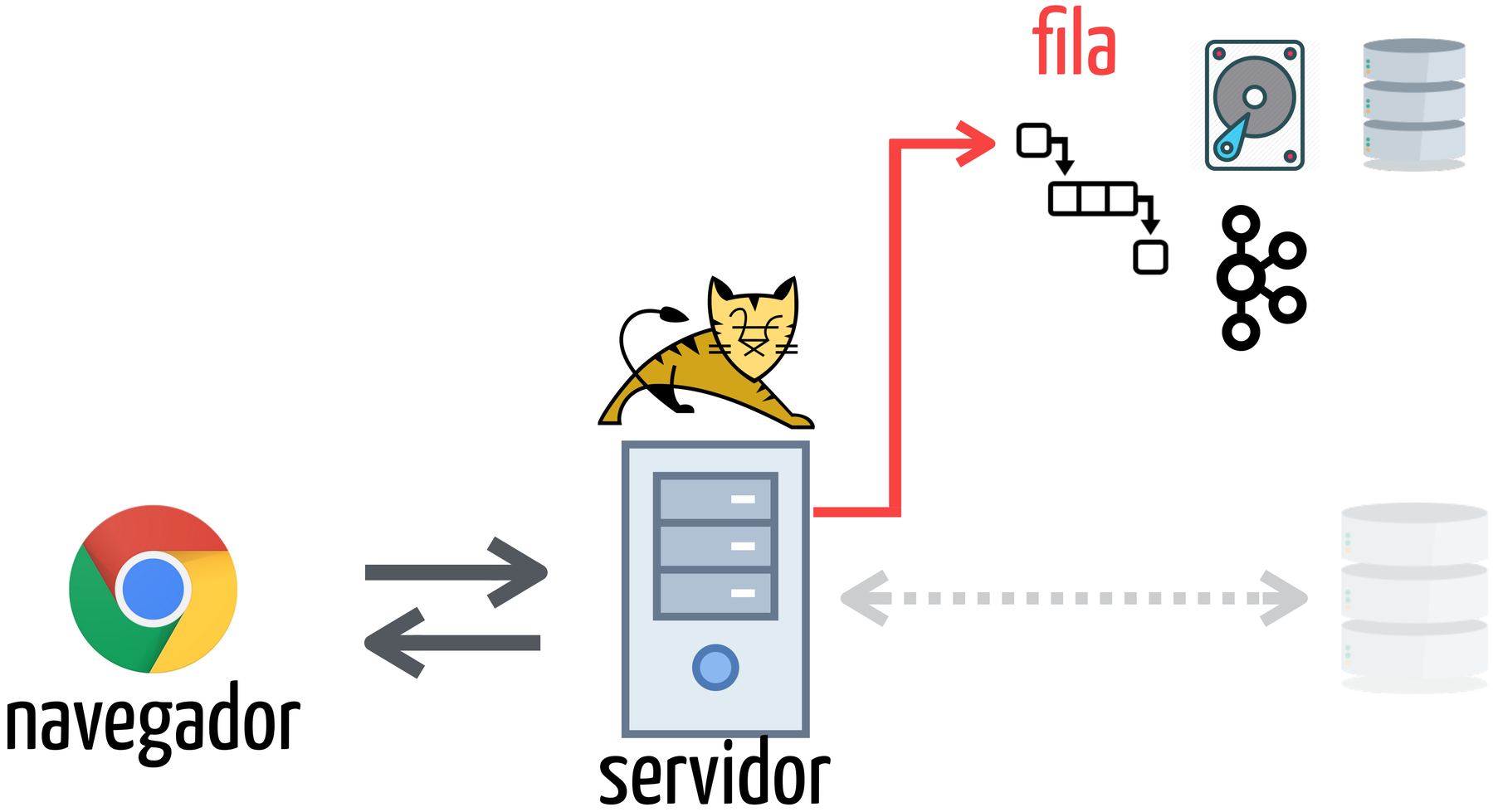

navegador
servidor
db

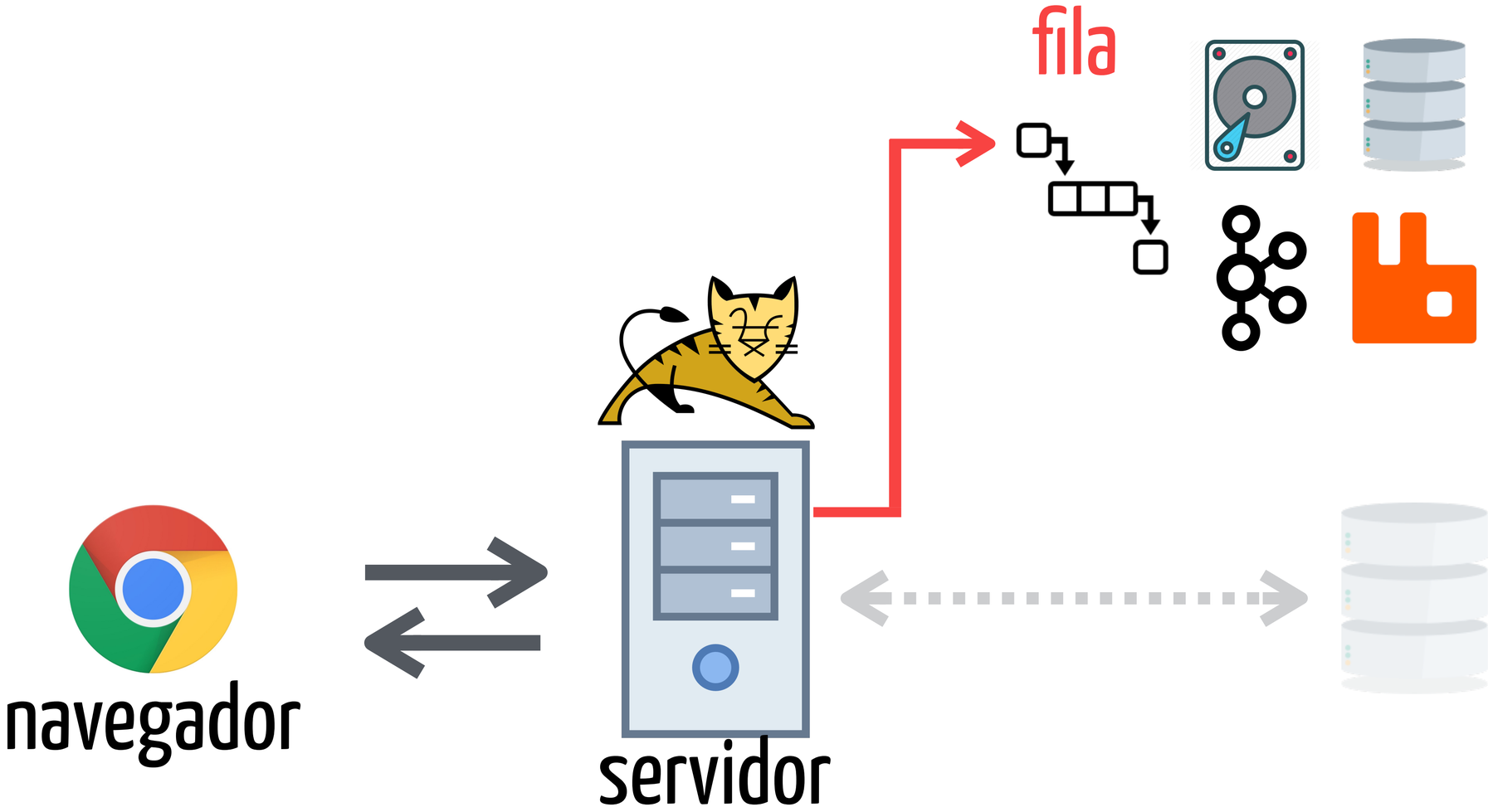

navegador
servidor
db


fila
navegador
servidor

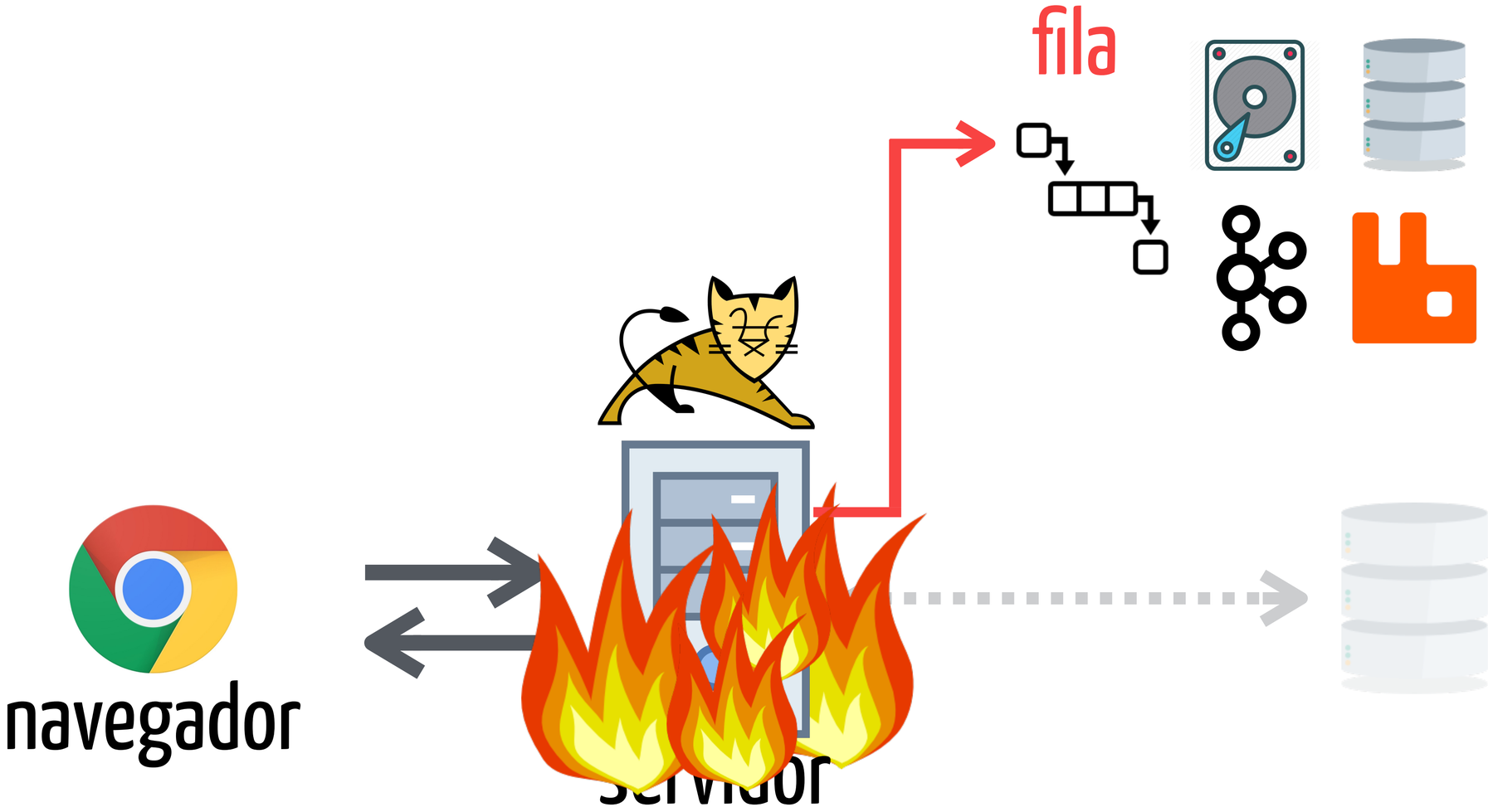

fila
navegador
servidor

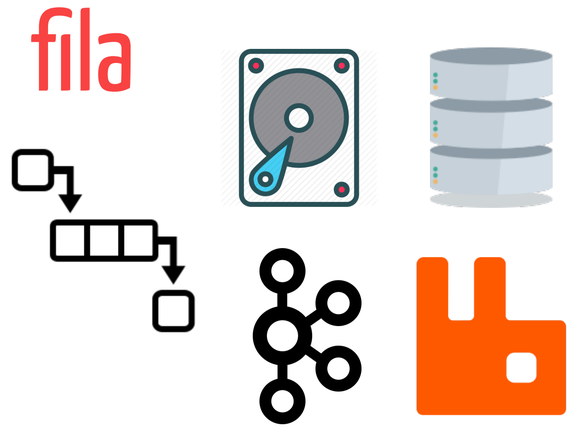

fila
navegador
servidor


fila
navegador
servidor

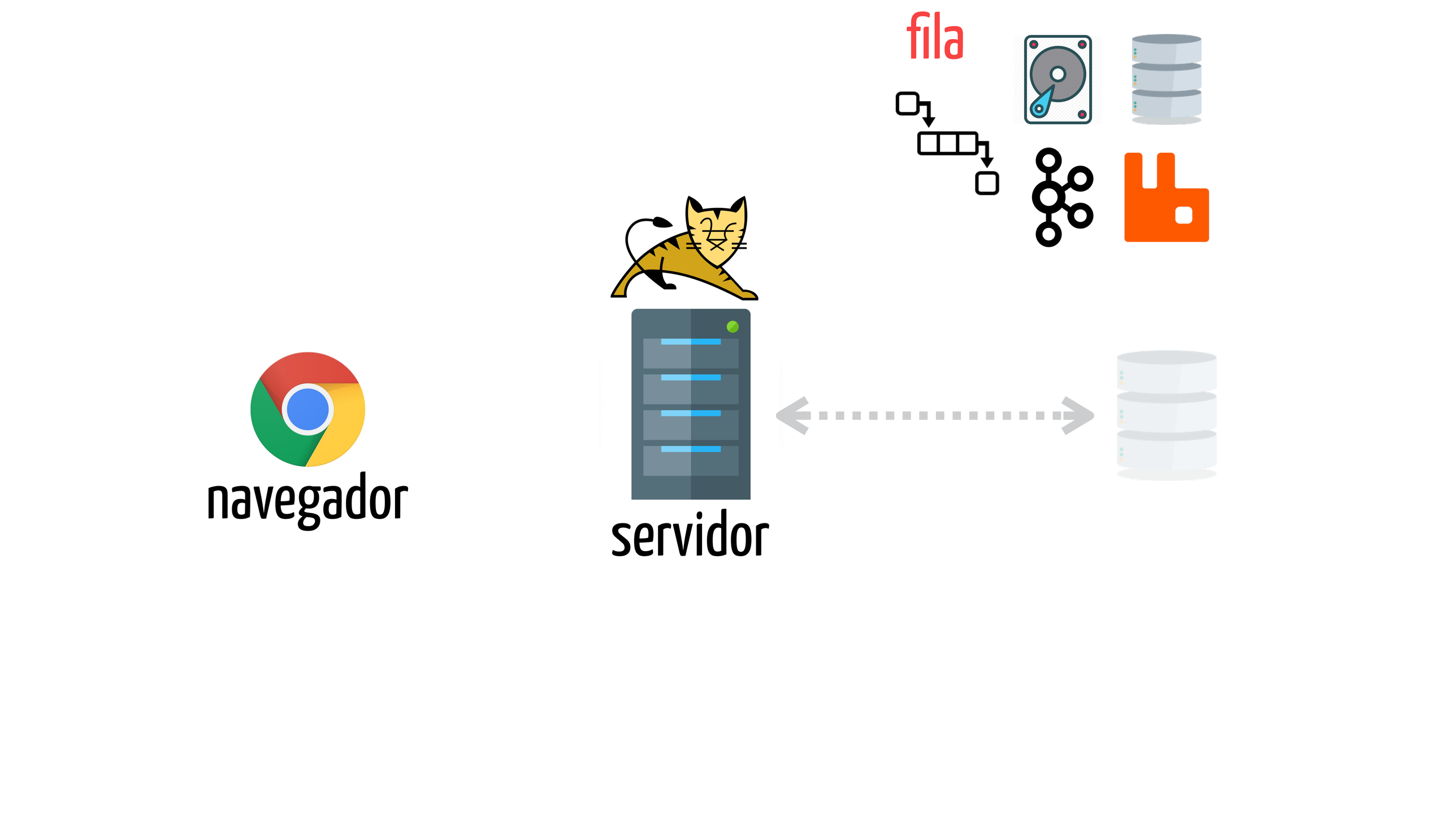

fila
navegador
servidor

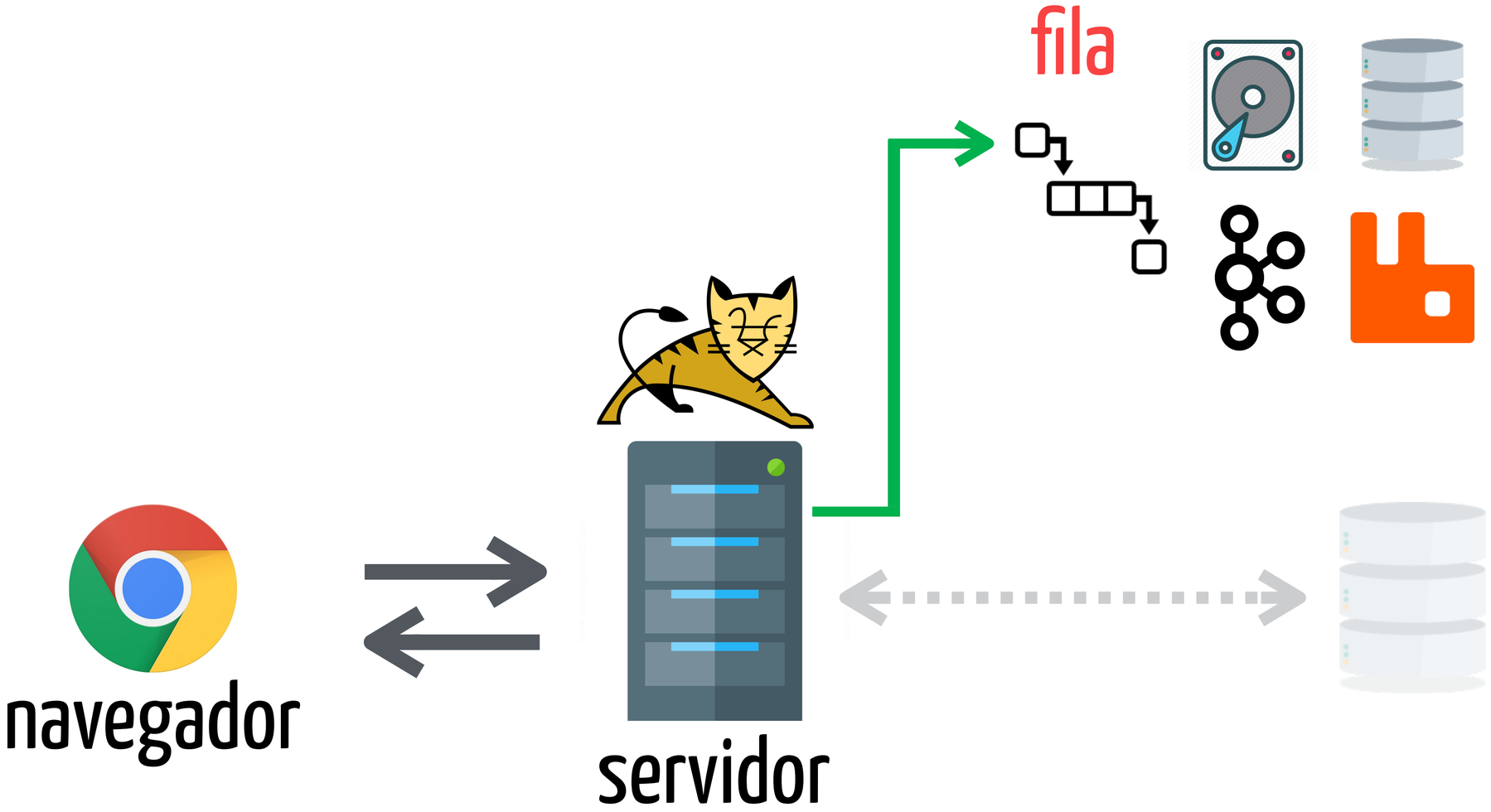
E mesmo que...


fila
navegador



navegador


fila
navegador
servidor


fila
navegador
servidor


RabbitMQ

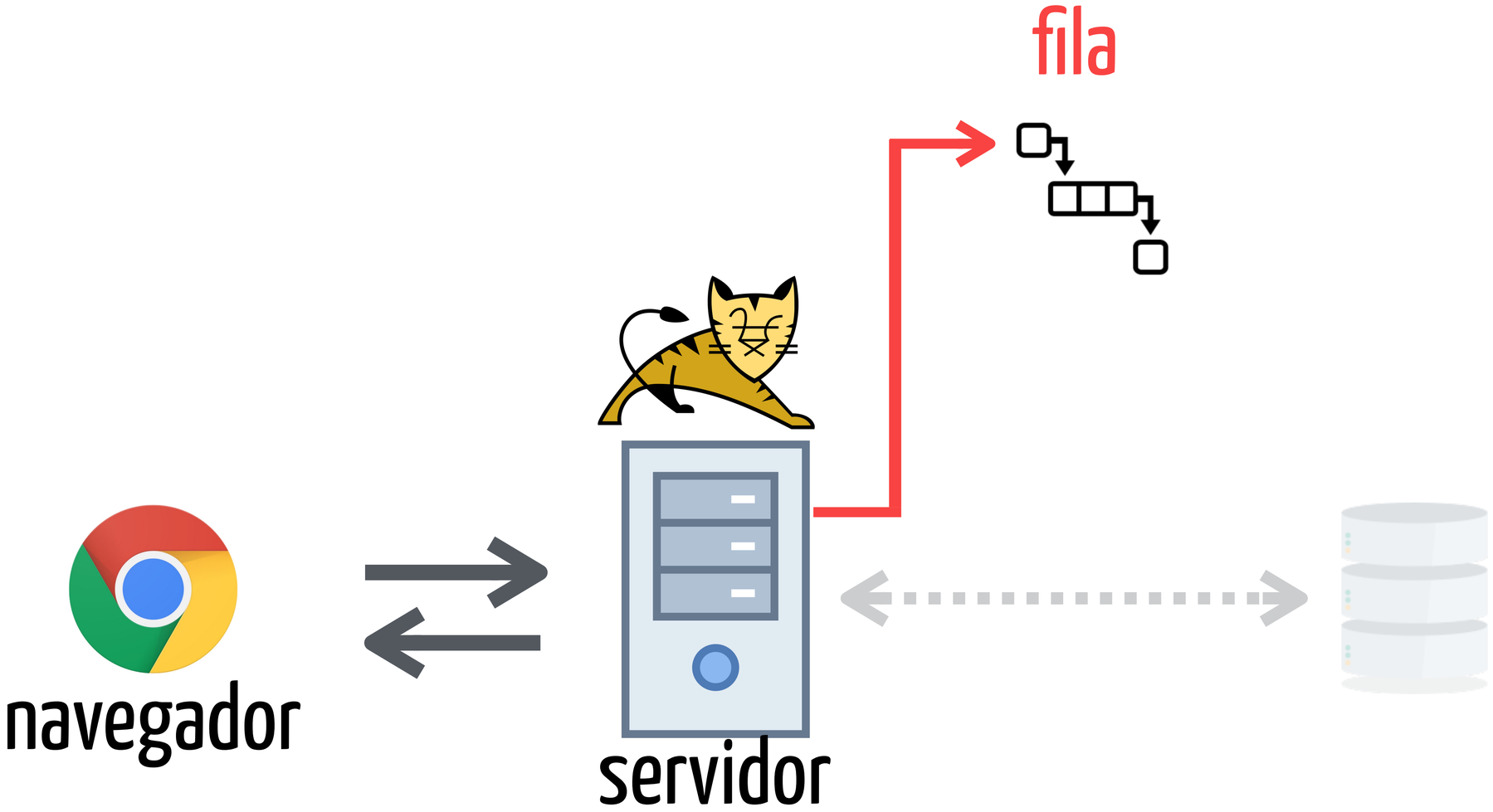
```java
@Controller
public class PedidoController {

    @Autowired
    public PedidoService service;

    @PostMapping("/vendas/finaliza")
    public void finalizaPedido(Pedido pedido) {
      // faz alguma validação de dados

      service.finaliza(pedido);
    }
}
```

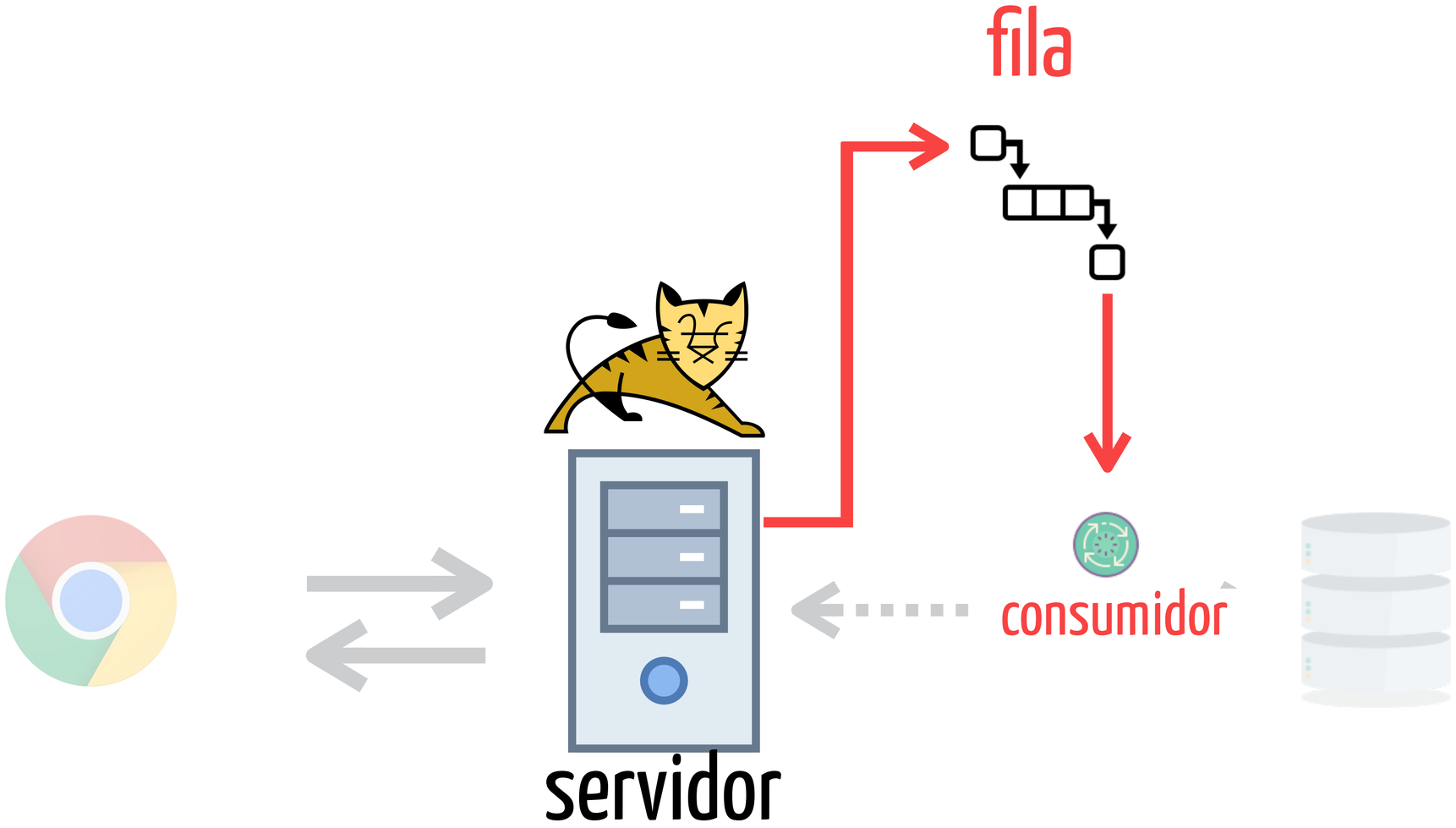
```java
@Controller
public class PedidoController {

    @Autowired
    public PedidoService service;

    @PostMapping("/vendas/finaliza")
    public void finalizaPedido(Pedido pedido) {
      // faz alguma validação de dados

      service.finaliza(pedido);
    }
}
```

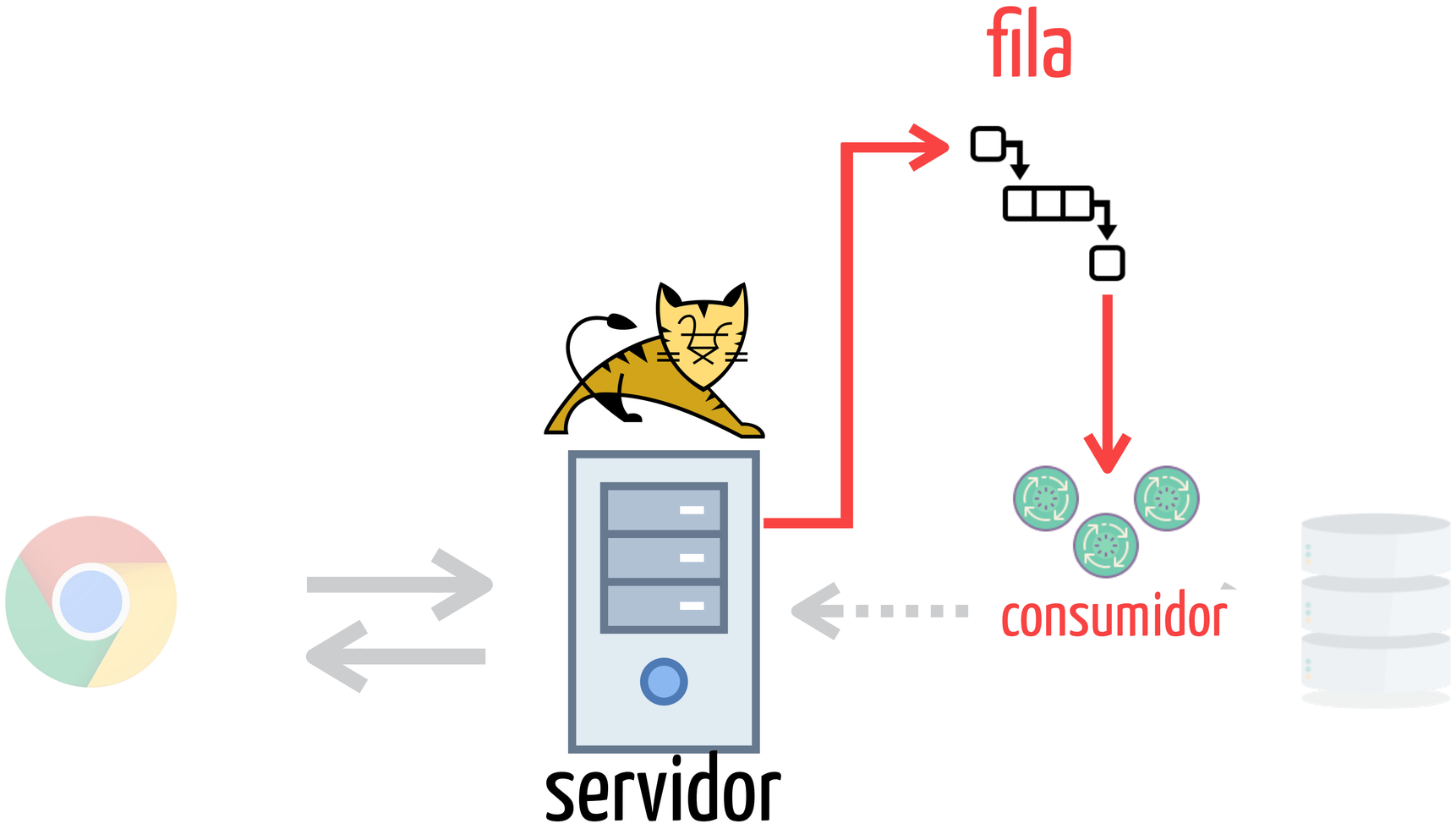
```java
@Controller
public class PedidoController {

    @Autowired
    public PedidoService service;

    @PostMapping("/vendas/finaliza")
    public void finalizaPedido(Pedido pedido) {
      // faz alguma validação de dados

      service.finaliza(pedido);
    }
}
```

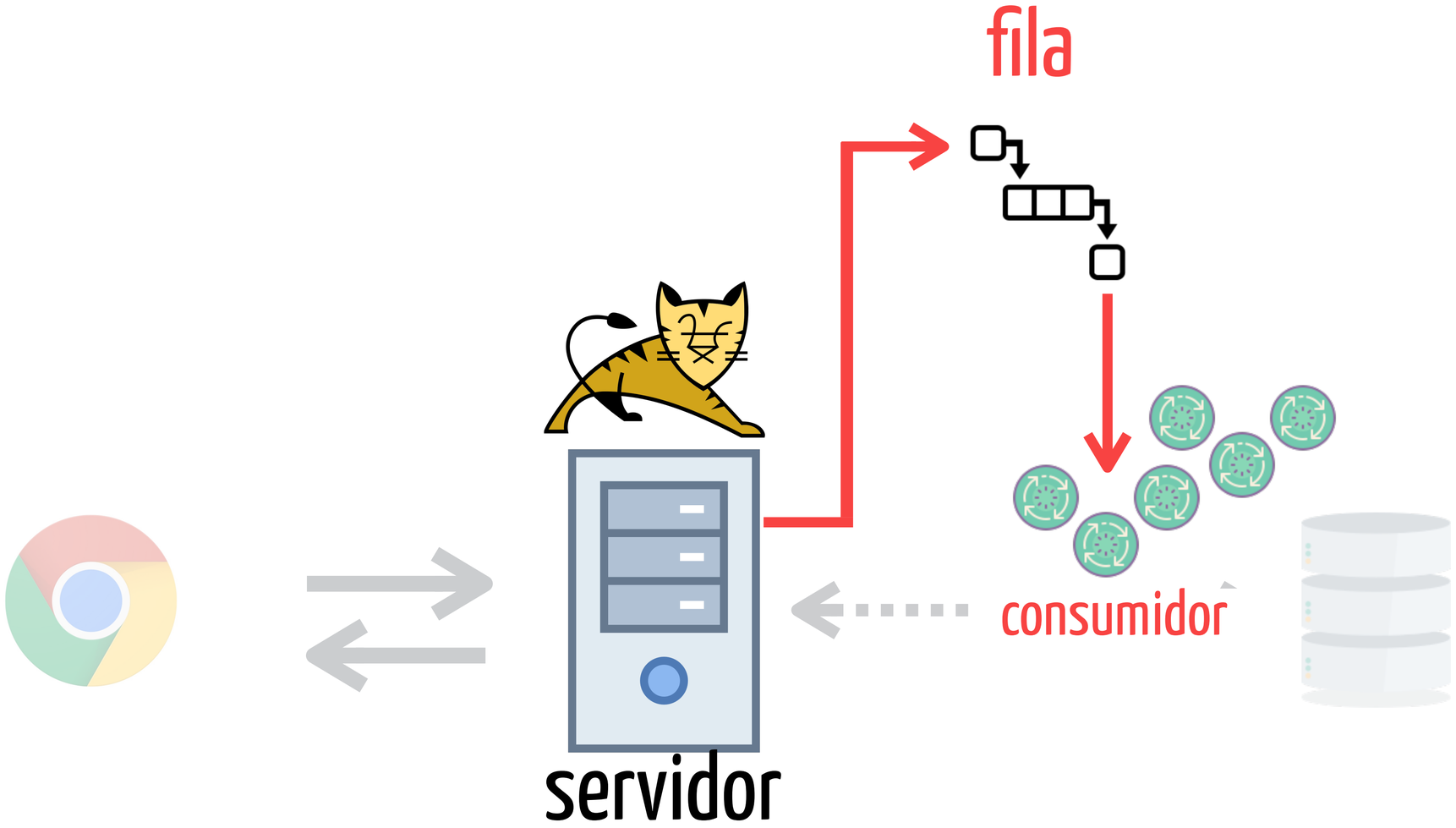
```java
@Controller
public class PedidoController {

    @Autowired
    public JmsTemplate fila;

    @PostMapping("/vendas/finaliza")
    public void finalizaPedido(Pedido pedido) {
      // faz alguma validação de dados

      fila.convertAndSend("fila.pedidos", pedido);
    }
}
```

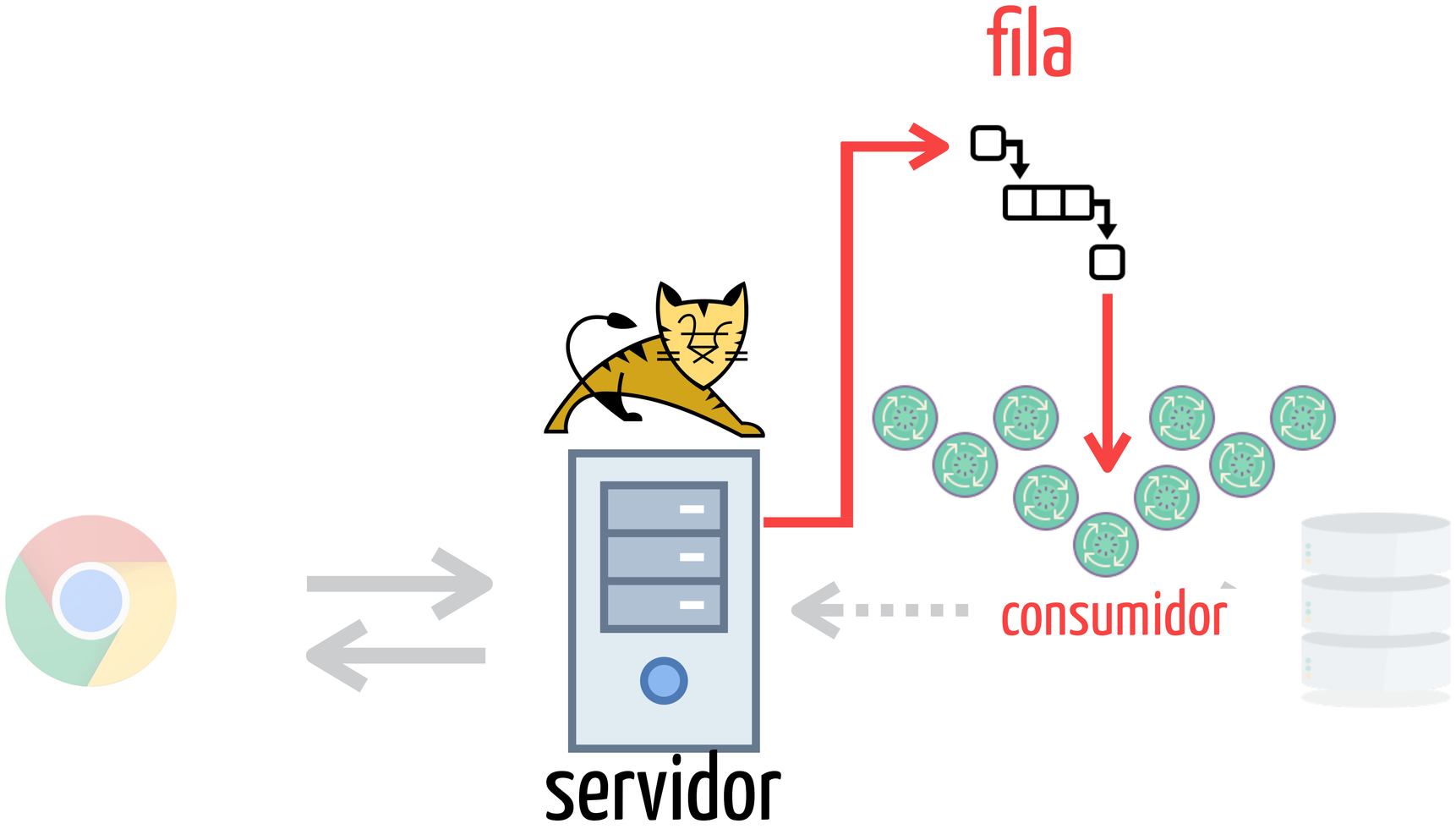
e em algum momento um processo consumiria a fila...

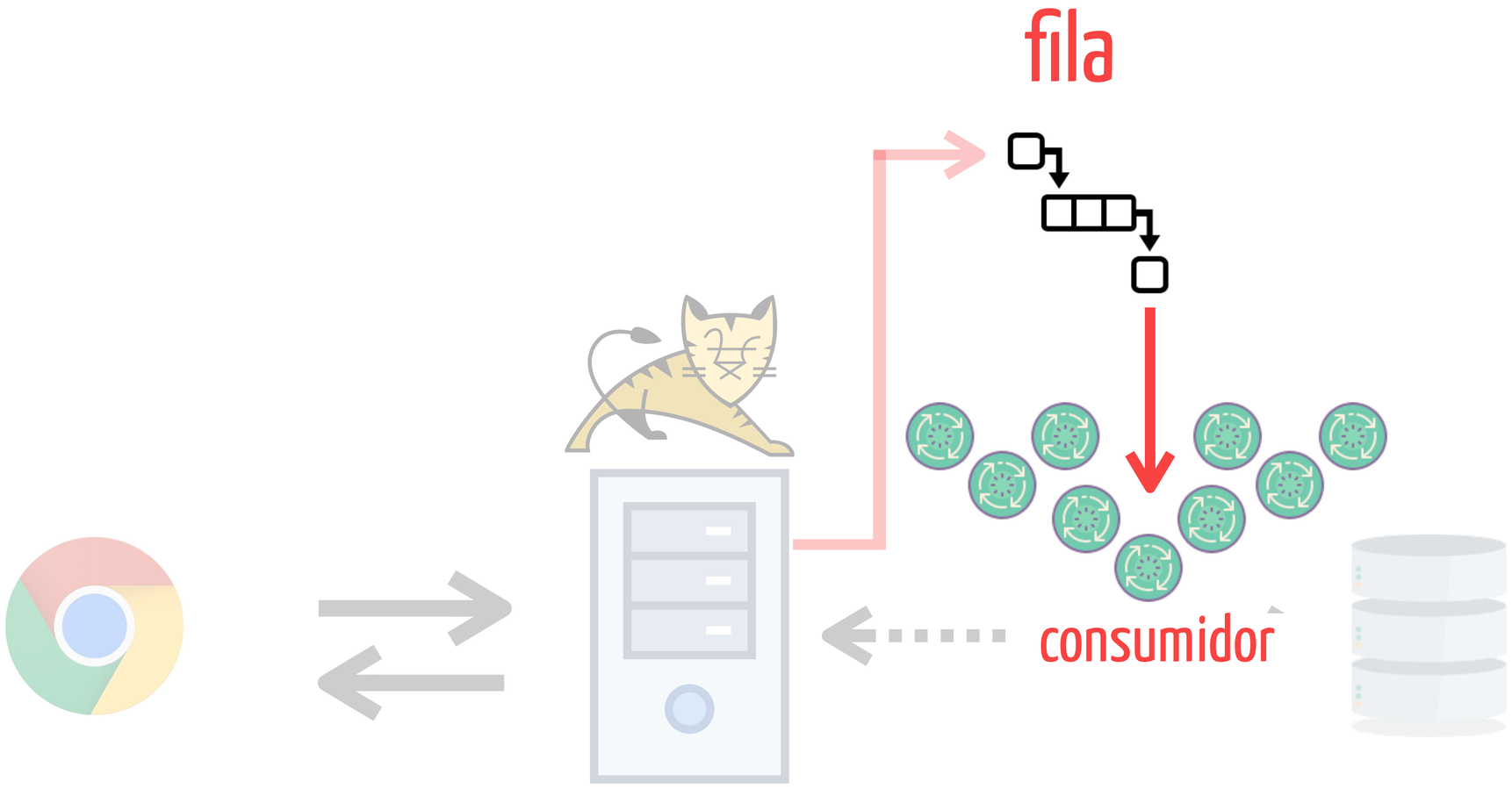

fila
navegador
servidor


fila
consumidor
servidor


fila
consumidor
servidor


fila
consumidor
servidor


fila
consumidor
servidor


fila
consumidor

```java
@Service
public class PedidoService {

    @Async
    public void finaliza(Pedido pedido) {
        // efetua pagamento no gateway
        // dá baixa no estoque
        // atualiza pedido
        // envia email de confirmação
        // gera nota fiscal
    }
}
```

```java
@Service
public class PedidoService {

    @Async
    public void finaliza(Pedido pedido) {
        // efetua pagamento no gateway
        // dá baixa no estoque
        // atualiza pedido
        // envia email de confirmação
        // gera nota fiscal
    }
}
```

```java
@Service
public class PedidoService {

    @Async
    public void finaliza(Pedido pedido) {
        // efetua pagamento no gateway
        // dá baixa no estoque
        // atualiza pedido
        // envia email de confirmação
        // gera nota fiscal
    }
}
```

```java
@Service
public class PedidoService {

    @JmsListener(destination="fila.pedidos")
    public void finaliza(Pedido pedido) {
        // efetua pagamento no gateway
        // dá baixa no estoque
        // atualiza pedido
        // envia email de confirmação
        // gera nota fiscal
    }
}
```

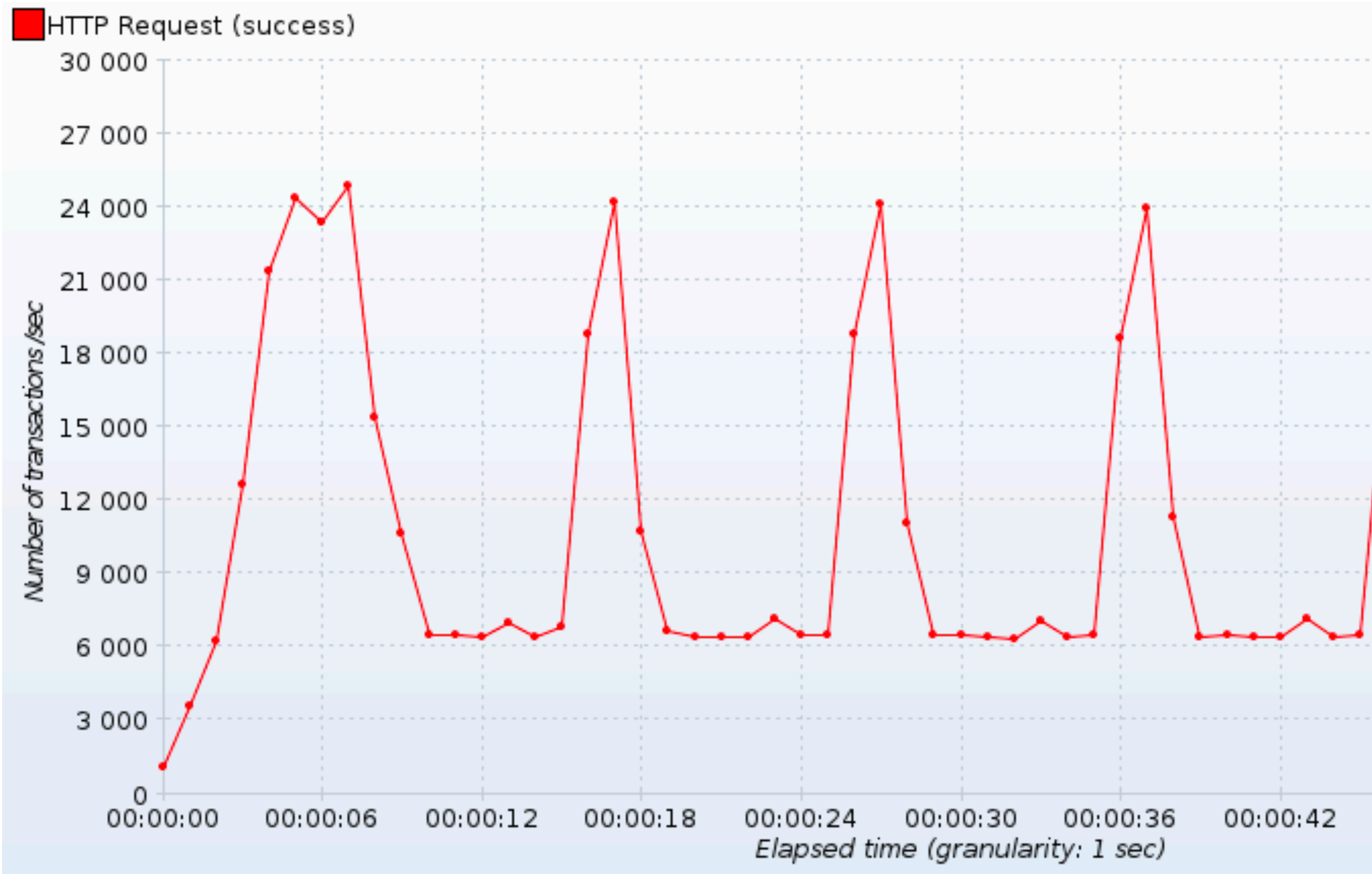
no fim nossa requisição
continuaria respondendo
imediatamente

```java
@Controller
public class PedidoController {

    @Autowired
    public JmsTemplate fila;

    @PostMapping("/vendas/finaliza")
    public void finalizaPedido(Pedido pedido) {
      // faz alguma validação de dados

      fila.convertAndSend("fila.pedidos", pedido);
    }
}
```

```java
@Controller
public class PedidoController {

    @Autowired
    public JmsTemplate fila;

    @PostMapping("/vendas/finaliza")
    public void finalizaPedido(Pedido pedido) {
      // faz alguma validação de dados

      fila.convertAndSend("fila.pedidos", pedido);
    }
}
```

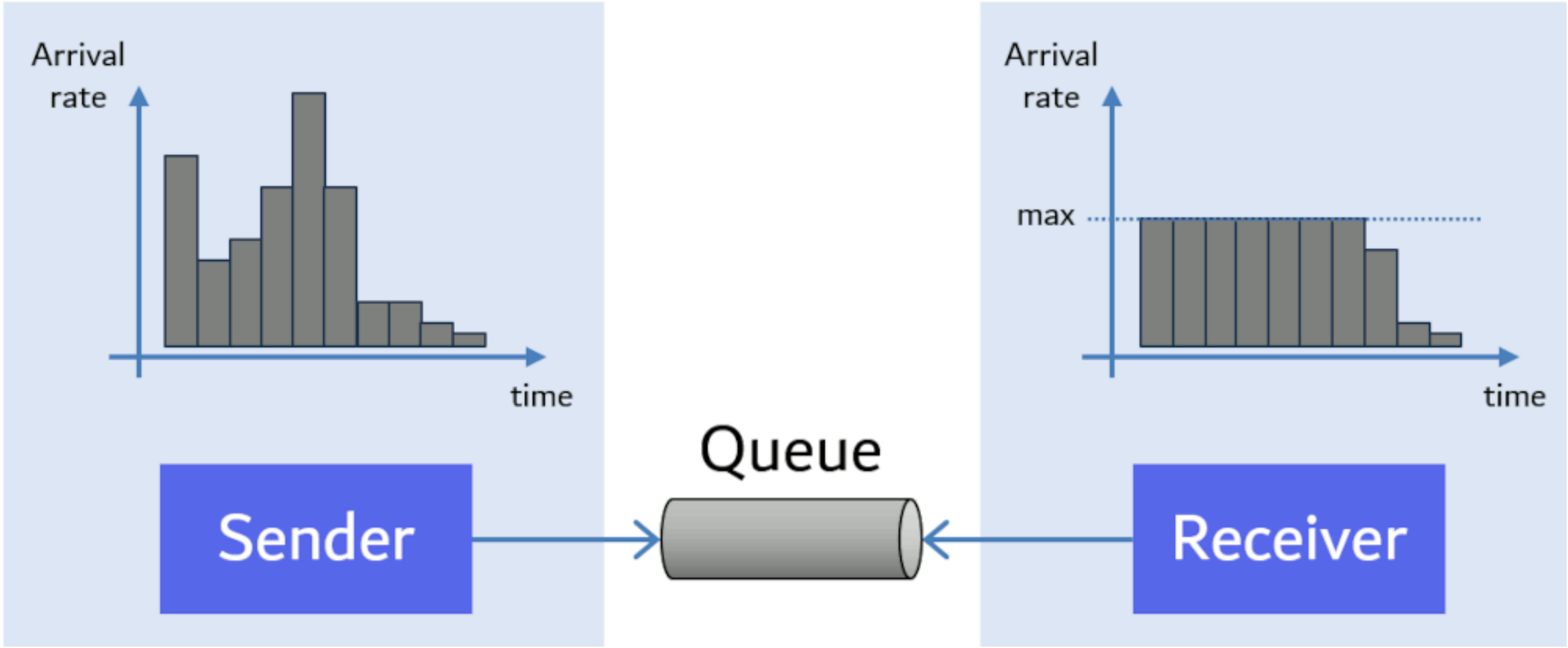
```java
@Controller
public class PedidoController {

    @Autowired
    public JmsTemplate fila;

    @PostMapping("/vendas/finaliza")
    public void finalizaPedido(Pedido pedido) {
      // faz alguma validação de dados

      fila.convertAndSend("fila.pedidos", pedido);
    }
}
```

IMEDIATO!


HTTP Request (success)
30 000
27 000
24 000
21 000
18 000
15 000
12 000
9 000
6 000
3 000
0
Number of transactions /sec
00:00:00
00:00:06
00:00:12
00:00:18
00:00:24
00:00:30
00:00:36
00:00:42
Elapsed time (granularity: 1 sec)

TRAFFIC SHAPING

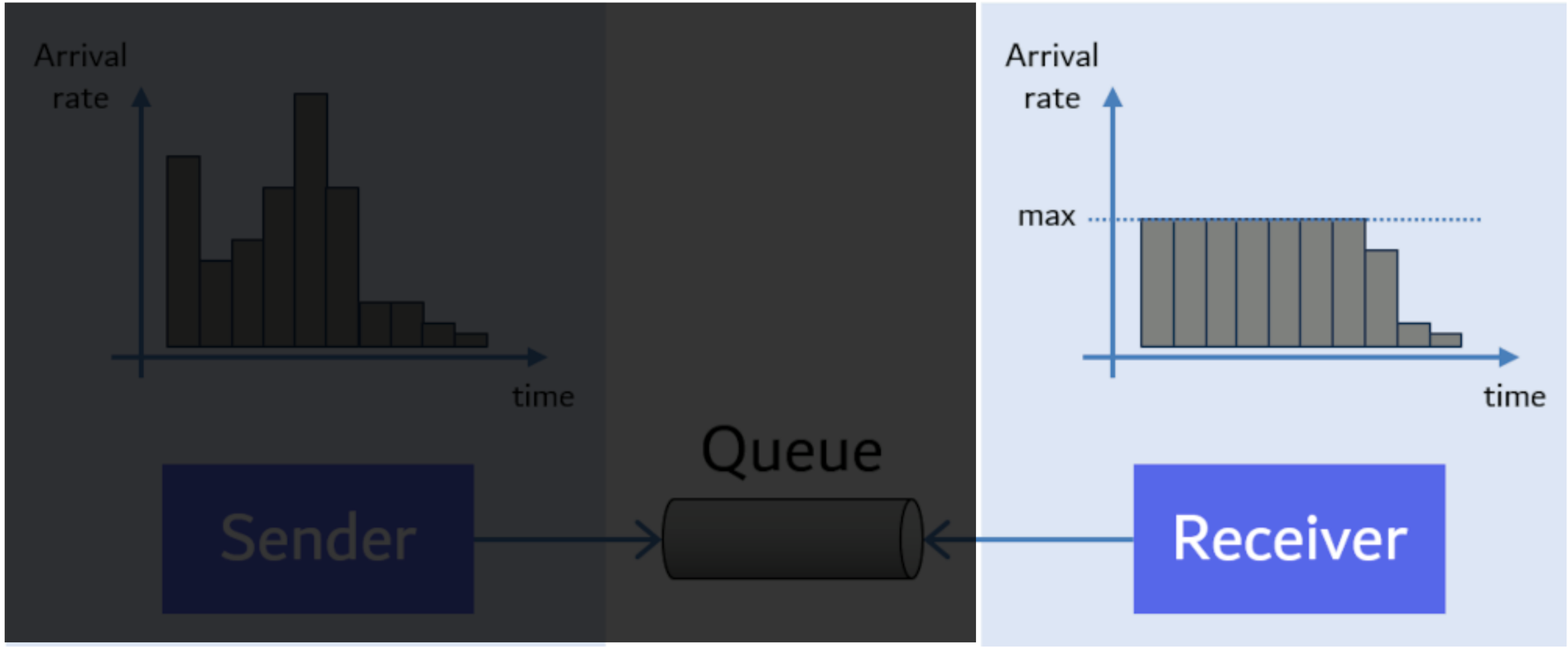
Filas invertem o fluxo de controle



https://www.enterpriseintegrationpatterns.com/ramblings/queues_flow_control.html

TRAFFIC SHAPING

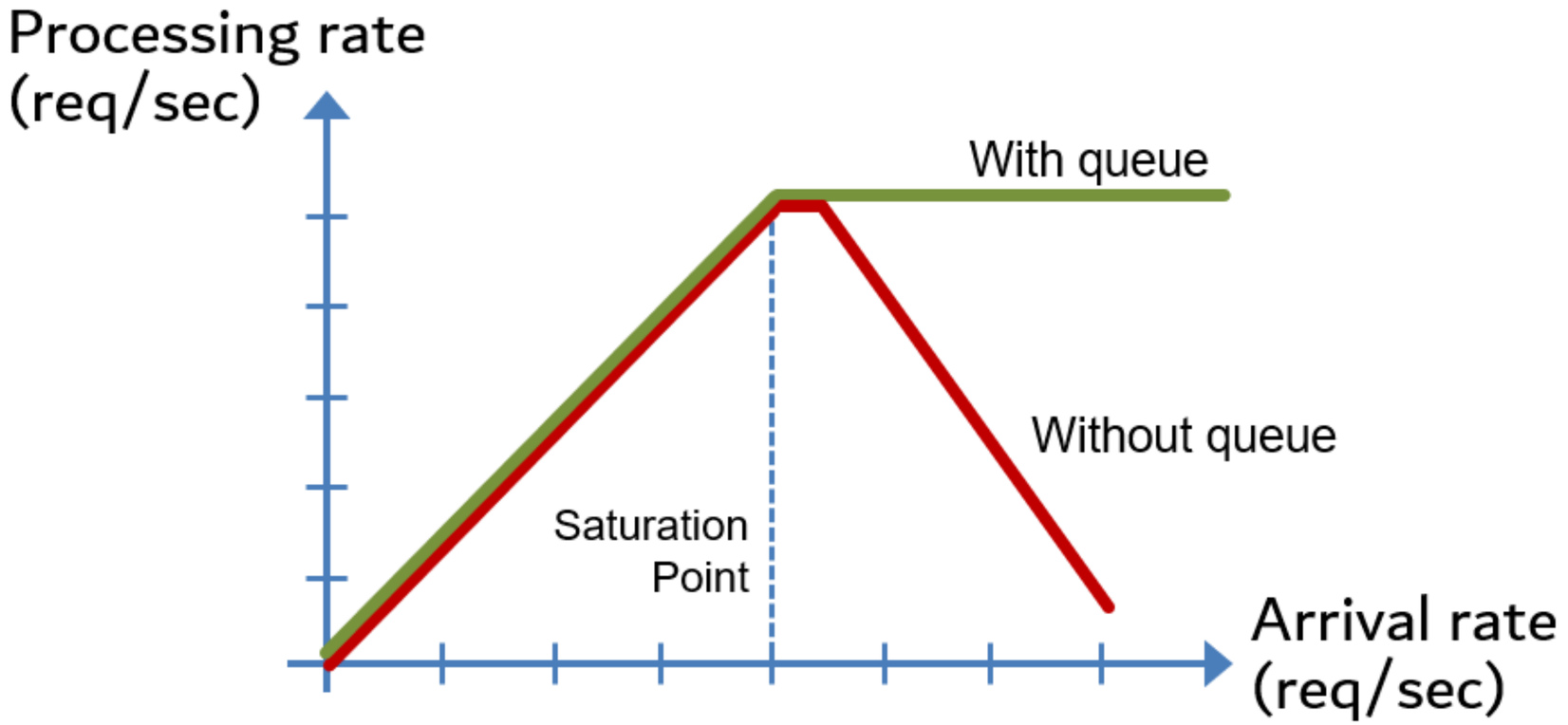
Filas invertem o fluxo de controle



https://www.enterpriseintegrationpatterns.com/ramblings/queues_flow_control.html

TRAFFIC SPIKES PROTECTION

Filas funcionam como um “buffer” para proteger o sistema de ruídos e picos



https://www.enterpriseintegrationpatterns.com/ramblings/queues_flow_control.html